Pontificia Universidad Javeriana
Facultad de Ingeniería — Ingeniería de Sistemas

Software Design Document
SDD

GammaLab v2.0

Proyecto	Plataforma de análisis y visualización de señales neurofisiológicas Gamma Lab
Documento	Software Architecture Document — SDD
Versión	2.0
Estado	En elaboración
Fecha	27 de agosto de 2026

Autores

Juan Felipe Romero Ortiz — `juanromeroo@javeriana.edu.co`
Laura Juliana Garzón Arias — `garzonlj@javeriana.edu.co`
Xiomara Herrera Suárez — `x_herrera@javeriana.edu.co`
Sebastián Almanza Galvis — `s.almanzag@javeriana.edu.co`

Nota de uso. Documento académico/interno del equipo de proyecto. No distribuir sin autorización.
Trazabilidad. Este SDD detalla la implementaci´on t´ecnica, estructura modular y dise˜no funcional de Gamma Lab, en coherencia con el SAD y el SRS previamente definidos.

Índice

1. Introducción **6**

1.1. Propósito del documento . . . 6

1.2. Alcance . . . 6

1.3. Audiencia . . . 6

2. Arquitectura **8**

2.1. Vista Lógica . . . 8

2.1.1. Propósito . . . 8

2.2. Vista Física . . . 10

2.2.1. Propósito . . . 10

2.2.2. Relaciones . . . 11

2.2.3. Versiones de las librerías . . . 11

2.3. Vista de Procesos . . . 11

2.3.1. Propósito . . . 11

2.4. Arquitectura de implementación . . . 13

2.4.1. Organización del código y reglas de dependencia . . . 13

2.4.2. Núcleo y ciclo de vida . . . 15

2.4.3. Arranque y cierre . . . 16

2.4.4. Hilos, tareas y recursos . . . 17

2.4.5. Fallos, registro y mensajes . . . 19

2.4.6. Extensibilidad . . . 20

3. Diseño Detallado **21**

3.1. Objetivo de la sección . . . 21

3.2. Estructura del Sistema . . . 22

3.2.1. Propósito . . . 22

3.2.2. Organización general . . . 22

3.2.3. Descripción de clases por componentes . . . 22

3.2.4. Resumen . . . 30

3.3. Comportamiento del Sistema . . . 30

3.3.1. Propósito . . . 30

3.3.2. Principales flujos del sistema . . . 30

3.4. Persistencia y Modelos de Datos . . . 31

3.4.1. Propósito . . . 31

3.4.2. Entidades de datos 31
3.5. Relaciones 32
3.6. Interfaz de Usuario 32
3.6.1. Proposito 32
3.6.2. Organización de la pantalla principal 33
3.6.3. Carga y guardado de archivos (pestaña *File*) 35
3.6.4. Preprocesamiento 37
3.6.5. Analysis 38

4. Comportamiento del sistema **44**
4.1. Diagrama guardar proyecto 44
4.2. Diagrama cerrar proyecto 44
4.3. Diagrama wavelet 47

Índice de figuras

1. Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de plugins. . 8
2. Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida. 10
3. Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema. . . 12
4. Diagrama de clases — componente ApplicationCore 23
5. Diagrama de clases — componente MainUI 25
6. Diagrama de clases — componente Plugins 26
7. Diagrama de clases — componente DataStore 26
8. Diagrama de clases — componente FileIO 27
9. Diagrama de clases — componente Export 28
10. Diagrama de clases — componente Measurement 28
11. Diagrama de clases — componente ProjectSession 29
12. Organización de la pantalla principal de Gamma Lab 33
13. opciones de slope 33
14. opciones de eliminar medida 34
15. opciones de zoom 34
16. explorador de archivos desplegado 34
17. resultados desplegados 35
18. Los resultados pueden ser exportados en formato csv. 35
19. Pestaña File de Gamma Lab 35
20. Flujo de carga de la señal (*Open Signal*) 36
21. Flujo de carga de proyecto (*Open Project*) 36
22. Pestaña Preprocessing de Gamma Lab 37
23. opciones de trials 37
24. opciones de remove artifact 38
25. opciones de filter 38
26. opciones de FFT 39
27. opciones de FFT Average 39
28. opciones de PSD 40
29. opciones de PSD Average 40
30. opciones de Average 41
31. opciones de Event related potential 41
32. opciones de Wavelet 42
33. opciones de Wavelet Average 42

34. opciones de PAC 43
35. Diagrama de secuencia para guardar proyecto 45
36. Diagrama de secuencia para cerrar proyecto 46
37. Diagrama de secuencia para wavelet 48

Índice de cuadros

1. Descripción de componentes de la Vista Lógica 8
2. Paquetes de primer nivel y componentes que alojan 13
3. Correspondencia ADR / principio → sección del SDD 14
4. Eventos publicados en el canal del Kernel 16
5. Severidades y tratamiento 19
6. Contrato `IPlugin` 20

1. Introducción

1.1. Propósito del documento

El presente **Software Design Document (SDD)** tiene como propósito documentar el diseño técnico de GammaLab v2.0, describiendo la organización de sus componentes, las responsabilidades de sus módulos, las relaciones entre ellos y los mecanismos que permiten su interacción. El documento profundiza en las decisiones de diseño que orientan la implementación del sistema, complementando la descripción arquitectónica establecida en el **Software Architecture Document (SAD)** y los requisitos definidos en el **Software Requirements Specification (SRS)**.

Asimismo, el SDD proporciona una referencia técnica para comprender la estructura interna del software, analizar sus flujos de ejecución y orientar el desarrollo, mantenimiento y validación de sus funcionalidades.

1.2. Alcance

El alcance de este documento comprende la descripción de la arquitectura y el diseño detallado de GammaLab v2.0, desde la organización general de sus componentes hasta los elementos técnicos que intervienen en su funcionamiento. Para ello, incluye los siguientes aspectos:

- **Arquitectura del sistema:** Descripción de las vistas de escenarios, lógica, de despliegue, de procesos y física, así como de los principales componentes y sus responsabilidades.
- **Estructura del sistema:** Documentación de las clases, atributos, métodos y responsabilidades asociadas a los componentes principales.
- **Comportamiento del sistema:** Descripción de los flujos de ejecución y de las interacciones entre los componentes durante las operaciones principales.
- **Persistencia y modelos de datos:** Identificación de las estructuras utilizadas para representar y almacenar las señales, los ensayos *(trials)* y otros datos relevantes del sistema.
- **Relaciones entre componentes:** Descripción de las dependencias, los mecanismos de comunicación y la utilización de los servicios compartidos.
- **Interfaz de usuario:** Descripción de la organización de la interfaz, sus elementos de navegación y las principales funcionalidades disponibles para la gestión, el preprocesamiento y el análisis de señales.

1.3. Audiencia

La audiencia principal de este documento incluye:

- **Desarrolladores de software:** Proporciona información sobre la estructura interna del sistema, las responsabilidades de las clases, las dependencias entre componentes y los criterios de diseño que orientan la implementación y el mantenimiento.
- **Equipo de QA:** Sirve como referencia para contrastar el diseño y el comportamiento del sistema con los requisitos especificados, así como para identificar posibles inconsistencias entre lo definido y lo implementado.

- **Directora de tesis:** Permite evaluar la coherencia técnica del diseño, su correspondencia con los requisitos establecidos y el avance de las funcionalidades contempladas para GammaLab v2.0.
- **Stakeholders:** Ofrece una descripción técnica general de la solución, sus principales componentes y las funcionalidades que contempla, facilitando la comprensión de su organización y evolución.+

2. Arquitectura

Esta sección consolida la arquitectura de GammaLab 2.0 desde la perspectiva del Software Design Document (SDD), alineada con el Software Architecture Document (SAD). Su propósito es ofrecer una visión clara de los componentes, relaciones, flujos y decisiones que soportan el diseño e implementación descritos posteriormente en el Diseño Detallado.

A continuación, se desarrollan las vistas específicas que articulan la arquitectura hacia el diseño e implementación del sistema.

2.1. Vista Lógica

2.1.1. Propósito

La vista lógica describe los componentes más importantes del sistema sin detallar a profundidad su instalación en hardware. Su objetivo es proporcionar una representación clara de cómo se organizan las responsabilidades y cómo interactúan los módulos fundamentales de GammaLab 2.0.

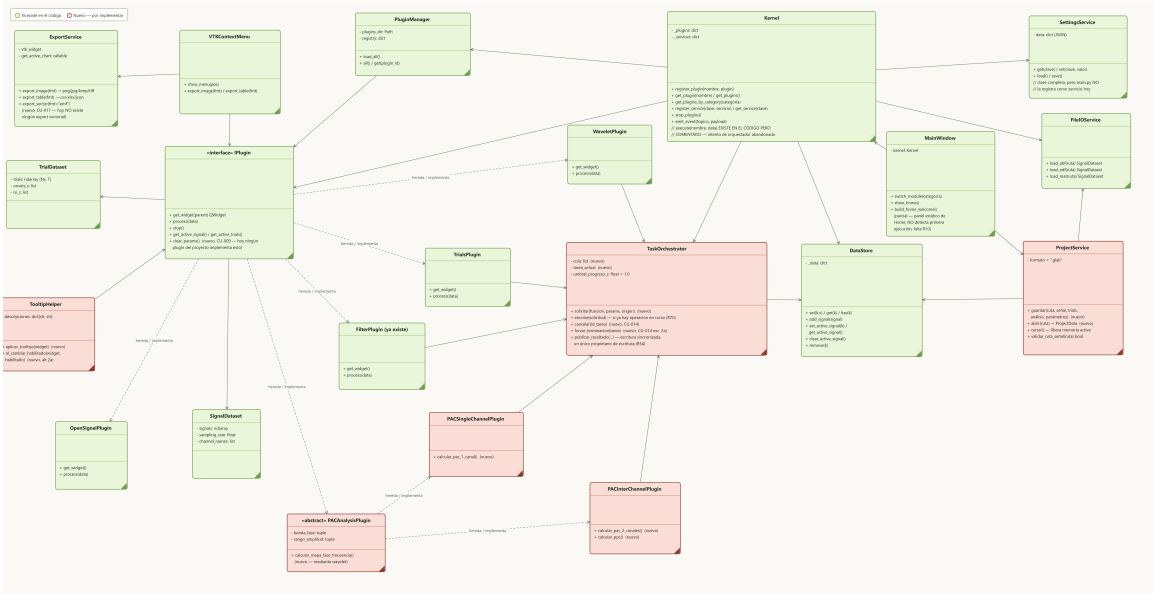


Figura 1: Vista lógica del sistema GammaLab 2.0: núcleo, servicios y familias de plugins.

La Figura 1 ilustra los principales componentes del software y sus interfaces. La Tabla 1 describe cada uno de ellos.

Cuadro 1: Descripción de componentes de la Vista Lógica

Componente	Descripción	Relaciones / Dependencias
Kernel (núcleo)	Orquestador central de la aplicación: administra el registro de plugins y servicios, coordina la ejecución y media la comunicación entre módulos.	Provee `register_service()`/`get_service()` y `register_plugin()`; es consumido por `MainWindow` y por todos los plugins.
PluginManager / `loader`	Descubre e instancia los plugins disponibles a partir de sus carpetas y archivos `properties.yml`.	Utiliza `IPlugin` como contrato; registra cada instancia en el Kernel.

Componente	Descripción	Relaciones / Dependencias
IPlugin (interfaz)	Contrato que toda extensión del sistema implementa (`initialize`, `start`, `stop`, `process`, `get_widget`).	Es implementada por todos los plugins; el Kernel la consume de forma polimórfica.
DataStore	Almacén centralizado en memoria de señales, trials, resultados intermedios y metadatos.	Consumido por el Kernel, los plugins y `FileIOService` mediante claves canónicas.
FileIOService	Lee archivos de señal en formato ABF, EDF y MAT, y los convierte a la estructura interna `SignalDataset`.	Utiliza las librerías `EEGReaders` (`pyabf`, `pyedflib`); publica el resultado en `DataStore`.
ExportService	Genera archivos de salida (imagen y tabla) a partir de las visualizaciones activas.	Utiliza `VTK` para capturar el render; es consumido por los plugins desde el menú contextual.
MeasurementService	Gestiona las mediciones interactivas de pendiente y amplitud sobre los gráficos.	Utiliza `VTK` para la interacción; persiste sus resultados en `DataStore`.
SettingsService	Administra la configuración general del sistema y las preferencias del usuario.	Persiste en `gamma_lab_settings.json`; consumido por el Kernel y los plugins.
ProjectService	Gestiona la persistencia del espacio de trabajo – señal activa, trials y análisis – en archivos de proyecto `.glab`.	Consume `DataStore` para serializar y restaurar el estado completo de la sesión.
TaskOrchestrator	Centraliza la ejecución de cómputo pesado (PAC, Wavelet, PSD) fuera del hilo de interfaz, con soporte de cancelación.	Es consumido por los plugins de análisis; se apoya en multiprocesamiento y multithreading.
Plugins (extensible)	Punto de extensión del sistema, organizados por categoría (E/S, Preprocesamiento, Análisis, Medición).	Consumen los servicios del Kernel; dependen de `PyQt5` y `VTK` para su interfaz.
MainWindow (APP)	Ventana principal de la aplicación; gestiona la navegación entre categorías de plugins y aloja su interfaz.	Consume el Kernel para descubrir y mostrar los plugins registrados.
PyQt5 (framework)	Framework de interfaz gráfica que administra ventanas, widgets y eventos del usuario.	Utilizado por el Kernel, `MainWindow` y los plugins para construir la interfaz.
VTK (librería)	Librería de visualización científica 2D/3D.	Usada directamente por los plugins de análisis y por `ExportService`/`MeasurementService`.
EEGReaders (librería)	Conjunto de librerías (`pyabf`, `pyedflib`) para leer formatos de señal biomédica.	Utilizado por `FileIOService`.
ScientificLibs (librería)	Conjunto de librerías científicas (NumPy, SciPy, PyWavelets) para el cómputo numérico.	Utilizado por los plugins de análisis y por `TaskOrchestrator`.

El *Kernel* centraliza el registro y consumo de servicios mediante `register_service()`/`get_service()`; el *DataStore* soporta la comunicación entre módulos mediante claves canónicas. El código se organiza en paquetes correspondientes a estas mismas responsabilidades (`io`, `preprocessing`, `analysis`, `measure`), lo que mantiene la correspondencia entre el diseño lógico y la implementación.

2.2. Vista Física

2.2.1. Propósito

La vista física representa cómo se despliega y ejecuta GammaLab 2.0. Presenta los componentes físicos más importantes del sistema, sobre los cuales se instalan los componentes de software.

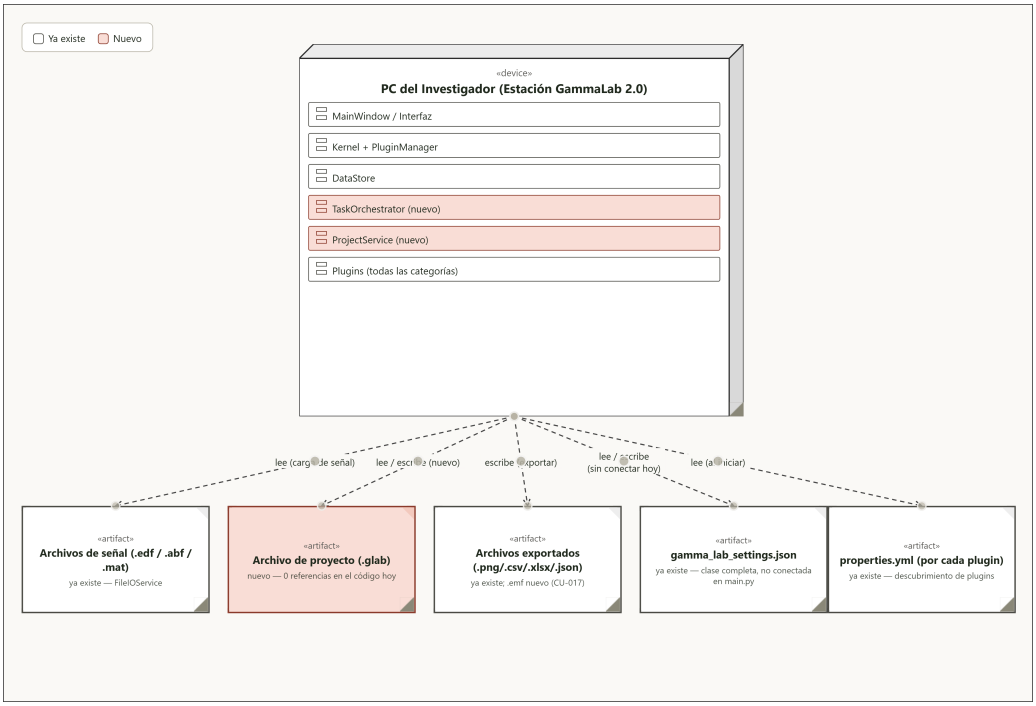


Figura 2: Vista física de GammaLab 2.0: estación única de escritorio y su superficie de entrada/salida.

La Figura 2 presenta la distribución física de los componentes del sistema en el entorno objetivo de ejecución. A continuación se describen los elementos principales:

- **«device» PC**: estación de trabajo del investigador donde se despliega y ejecuta el sistema completo; no existe arquitectura cliente-servidor.
- **«OS» Windows 10/11**: sistema operativo base; versión mínima Windows 10 de 64 bits.
- **«executionEnvironment» Python Runtime 3.11**: entorno de ejecución que sostiene la aplicación y sus librerías; versión mínima 3.11.x.
- **«artifact» GammaLab.exe**: ejecutable principal, generado con PyInstaller, que inicia la aplicación y coordina la carga de todos los componentes.
- **«artifact» Plugins (*.py)**: archivos de plugin cargados dinámicamente por el Kernel durante la ejecución.
- **«artifact» Archivos de señal** (`.edf`/`.abf`/`.mat`): leídos por `FileIOService`.
- **«artifact» Archivo de proyecto** (`.glab`): generado y leído por `ProjectService`.
- **«artifact» gamma_lab_settings.json**: persistencia de configuración gestionada por `SettingsService`.
- **«artifact» properties.yml** (por cada plugin): sostiene el descubrimiento automático de plugins.

2.2.2. Relaciones

- La relación *loads* indica que el ejecutable principal carga y ejecuta los archivos de plugin durante el tiempo de ejecución.
- La relación *uses* especifica que el ejecutable principal utiliza los servicios y librerías desplegados dentro del mismo entorno de ejecución.

2.2.3. Versiones de las librerías

Las versiones ancladas para las librerías utilizadas, tal como se fijan en `requirements.txt`, son las siguientes:

- **PyQt5** = Versión 5.15.11 (`PyQt5-Qt5` 5.15.2)
- **ScientificLibs**
  - NumPy = Versión 2.3.4
  - Pandas = Versión 2.3.3
  - SciPy = Versión 1.16.2
  - PyWavelets = Versión 1.9.0
- **VTK** = Versión 9.5.2
- **EEGReaders**
  - pyABF = Versión 2.3.8
  - pyEDFlib = Versión 0.1.42
- **Otras**
  - h5py = Versión 3.15.1
  - PyYAML = Versión 6.0.3
  - PyQtWebEngine = Versión 5.15.7

2.3. Vista de Procesos

2.3.1. Propósito

La vista de procesos describe el comportamiento dinámico del sistema completo como un único flujo maestro de sesión, construido a partir de las cadenas de precondición entre los casos de uso del sistema (p. ej. generar trials exige una señal previamente cargada).

El flujo de la Figura 3 se organiza en las siguientes etapas:

1. **Inicio.** El sistema registra los servicios del núcleo, incluido el `TaskOrchestrator`, y descubre los plugins disponibles a partir de sus archivos `properties.yml`.
2. **Apertura o carga.** El investigador abre un archivo de proyecto `.glab` o carga una señal desde la pantalla *Home*; la señal activa se publica en `DataStore`.
3. **Preprocesamiento.** Generación y edición de trials, selección de trials, remoción de artefactos, filtrado y ajuste de la frecuencia de muestreo.

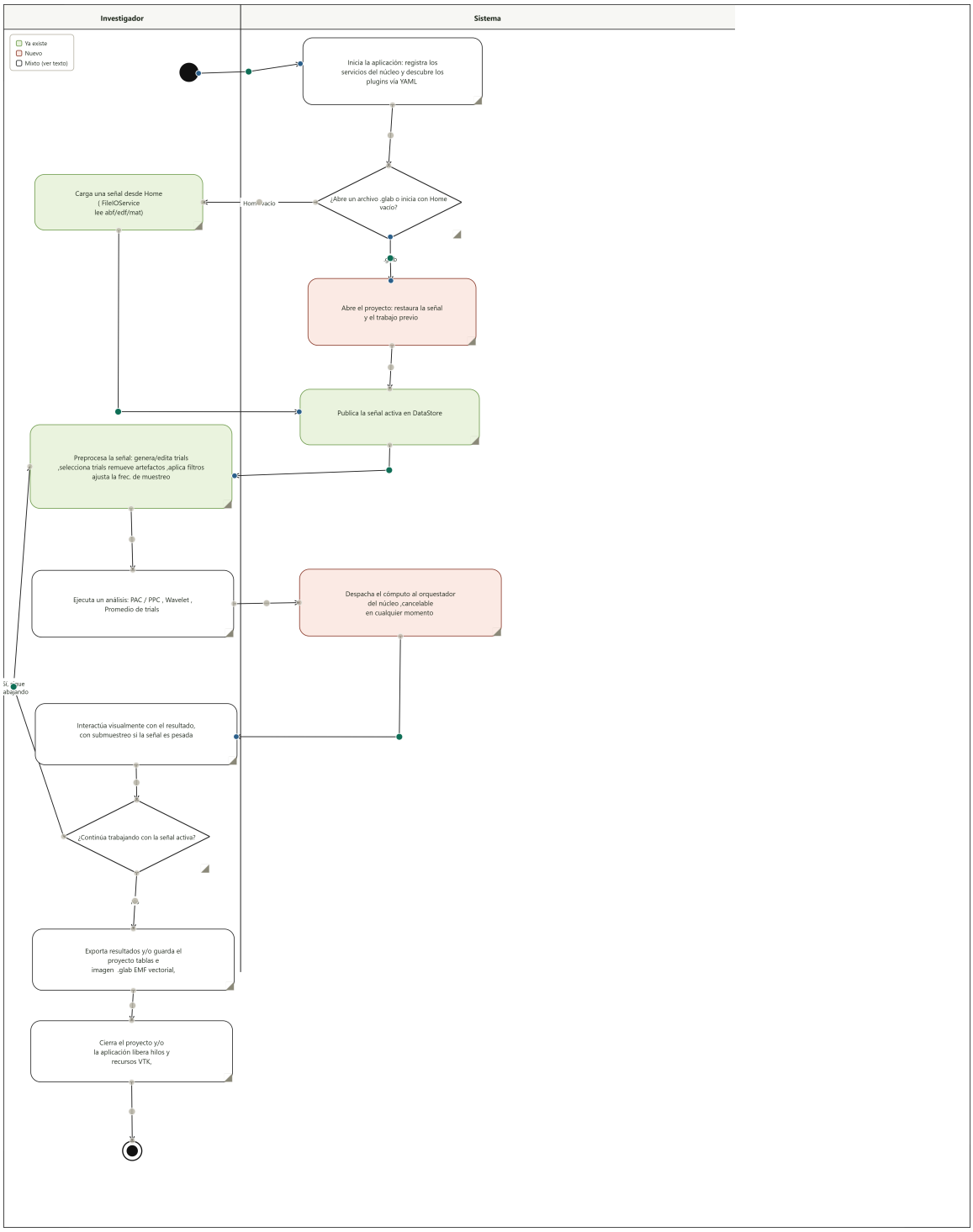


Figura 3: Flujo maestro de sesión de GammaLab 2.0, con carriles Investigador/Sistema.

4. **Análisis.** Cálculo de acoplamiento fase-amplitud (PAC) de un canal o cruzado entre canales, transformada Wavelet y promedio de trials.
5. **Orquestación.** Todo cómputo pesado se despacha al `TaskOrchestrator` del núcleo, que lo ejecuta fuera del hilo de interfaz y permite su cancelación en cualquier momento.
6. **Visualización.** Interacción visual con el resultado, con submuestreo automático cuando la señal es de gran tamaño.
7. **Iteración o cierre.** El investigador retorna al preprocesamiento para continuar el análisis, o exporta y guarda el proyecto mediante `ProjectService` antes de cerrar la sesión.

La navegación entre secciones, la limpieza de formularios y la ayuda contextual mediante `TooltipHelper` son transversales a toda la sesión: no ocupan un lugar fijo en la secuencia y se documentan como notas adjuntas al diagrama en vez de forzarse dentro del flujo principal.

2.4. Arquitectura de implementación

Las tres vistas anteriores describen el sistema a nivel de componentes. Esta sección baja al nivel en que se escribe el código: qué paquete puede importar a cuál, quién gobierna el ciclo de vida de las extensiones, en qué orden arranca y se cierra la aplicación, cómo se reparte el trabajo entre hilos, cómo se tratan los fallos y qué hay que hacer para añadir una extensión nueva.

Cada apartado se cierra con el estado real de implementación. El Software Architecture Document (SAD) prescribe trece decisiones de arquitectura (ADR-01 a ADR-13); varias están implementadas, otras lo están en parte y otras todavía no. Documentar como existente lo que no se ha construido convertiría este documento en una descripción de un sistema imaginario, así que la distinción se mantiene explícita en cada caso.

2.4.1. Organización del código y reglas de dependencia

El código se organiza en cuatro paquetes de primer nivel. La Tabla 2 indica qué componente de la Vista Lógica (Sección 2.1) vive en cada uno.

Cuadro 2: Paquetes de primer nivel y componentes que alojan

Paquete	Contenido	Componentes del SAD
`core/`	Núcleo, contrato de extensión, servicios compartidos, modelos de datos y algoritmos que cruzan fronteras de extensión.	Kernel, PluginManager, IPlugin, DataStore, FileIOService, ExportService, MeasurementService, SettingsService, ProjectService, TaskService.
`app/`	Capa de presentación: ventana principal, pantalla de bienvenida y widgets propios.	MainWindow.
`plugins/`	Las extensiones, agrupadas por categoría (`io`, `preprocessing`, `analysis`, `measure`, `faq`).	Plugins.
`test/`	Pruebas automatizadas, con la misma división por responsabilidad (`services_test`, `plugins_test`, `ui_test`, `performance_test`).	—

Dentro de `core/` la división es por responsabilidad: `core/kernel.py` (el núcleo), `core/plugins/` (contrato, descubrimiento y registro), `core/services/` (los servicios compartidos), `core/model/` (las estructuras de datos), `core/filters/` (los algoritmos compartidos) y `core/utils/` (utilidades transversales).

Reglas de dependencia. Son tres, y se enuncian en orden de importancia.

1. **`core/` no importa de `app/` ni de `plugins/`.** Es la regla que sostiene el estilo microkernel: el núcleo tiene que poder existir sin ninguna extensión concreta y sin la ventana principal. Si se invierte, el núcleo deja de ser un núcleo y pasa a ser una capa más del monolito.
2. **Las extensiones no se importan entre sí.** Lo que dos extensiones comparten sube a `core/`; lo que usa una sola se queda en su carpeta. Es la regla que hizo que `gabor.py` pasara del plugin de PAC a `core/filters/` cuando PPC empezó a necesitarlo, y la que mantiene a la wavelet de Morse dentro de PAC, porque solo PAC la usa.
3. **El cómputo no importa de Qt ni de VTK.** El módulo `compute.py` de cada extensión recibe arreglos de NumPy y devuelve arreglos de NumPy. Es lo que permite ejecutarlo en un hilo trabajador y probarlo sin levantar una interfaz gráfica (ADR-04).

Cómo se verifica. La primera regla se comprueba automáticamente: basta recorrer los archivos de `core/` y buscar importaciones de `app` o `plugins`. Al momento de redactar esta sección el resultado es **cero importaciones**, frente a 3 de `app/` hacia los otros paquetes y 32 de `plugins/`, que es la dirección correcta. La tercera regla se comprueba de la misma forma sobre cada `compute.py`, y está cubierta por pruebas: `test/services_test/test_gabor_en_core.py` verifica con el módulo `ast` que `core/filters/gabor.py` no importe nada de `plugins`, mirando las sentencias de importación y no el texto, porque los comentarios sí nombran a `plugins` y eso es documentación, no acoplamiento.

Correspondencia con las decisiones del SAD. La Tabla 3 relaciona cada decisión de arquitectura con el apartado de este documento que la desarrolla y con su estado de implementación.

Cuadro 3: Correspondencia ADR / principio → sección del SDD

ADR	Princ.	Decisión	Sección	Estado
ADR-01	P1	Estilo microkernel con núcleo propio	2.4.1, 2.4.2	Implementada
ADR-02	P1	Ciclo de vida como máquina de estados gobernada por el Kernel	2.4.2	**No implementada**
ADR-03	P2	Catálogo de eventos con carga útil tipada	2.4.2	**No implementada**
ADR-04	P3	Cómputo en un módulo sin dependencias de Qt ni VTK	2.4.1, 2.4.6	Parcial (9 de 19 extensiones)
ADR-05	P4	Descubrimiento declarativo validado	2.4.2, 2.4.6	Parcial
ADR-06	P5	`DataStore` con escritor único	2.4.4	Parcial
ADR-07	P6	Estrategia de memoria	2.4.4	Parcial
ADR-08	P7	Liberación determinista de recursos	2.4.3, 2.4.4	Parcial
ADR-09	P8	TaskOrchestrator híbrido (hilos y procesos)	2.4.4	Parcial (solo hilos, por medición)
ADR-10	P9	Control de admisión por estimación de costo	2.4.4	**No implementada**

ADR	Princ.	Decisión	Sección	Estado
ADR-11	P10	Manejador global de excepciones	2.4.5	**No implementada**
ADR-12	P11	Registro estructurado con archivo rotativo	2.4.5	**No implementada**
ADR-13	—	Python, PyQt5 y VTK como base	2.2	Implementada

2.4.2. Núcleo y ciclo de vida

Kernel. Es un `QObject` con cuatro responsabilidades y ninguna más: mantener el diccionario de extensiones registradas, mantener el diccionario de servicios, exponer un canal de eventos y recorrer las extensiones para detenerlas. No calcula, no dibuja y no lee archivos. Su interfaz pública es:

- `register_plugin(name, plugin)` — registra la extensión y le llama `initialize(kernel)`;
- `get_plugin(name)`, `get_plugins()`, `get_plugins_by_category(cat)` — consulta;
- `register_service(key, service)` / `get_service(key)` — el localizador de servicios;
- `emit_event(topic, payload)` — publicación de eventos;
- `stop_plugins()` — detención ordenada.

PluginManager. Recorre el árbol de `plugins/` buscando archivos `properties.yml`, instancia la clase declarada en cada uno y guarda el par `(meta, instancia)` en un registro indexado por el identificador de la extensión. El descubrimiento es tolerante a fallos: una carpeta cuyo descriptor esté mal formado, o cuya clase no pueda importarse, se omite con un mensaje en consola y el resto de la aplicación sigue arrancando.

Estados y transiciones. Hoy el ciclo de vida de una extensión es una secuencia, no una máquina de estados: *descubierta* (el `PluginManager` la instancia) → *inicializada* (el Kernel le llama `initialize`) → *activa* (`start` desde `MainWindow` al entrar en su pestaña, que pone el indicador `started`) → *detenida* (`stop`).

> **Brecha respecto al SAD (ADR-02).** El SAD prescribe seis estados explícitos — descubierta, inicializada, activa, inactiva, detenida y **fallida** — con las transiciones gobernadas únicamente por el núcleo. La implementación actual no mantiene el estado de cada extensión: el único indicador es el booleano `started` de `IPlugin`, las transiciones las dispara `MainWindow` y no `Kernel`, y no existe el estado *fallida*: una extensión cuya carga falla se omite en silencio y no aparece en la interfaz, en lugar de aparecer marcada como fallida. Cerrar esta brecha es un trabajo acotado — un campo de estado en el Kernel y una transición por método — pero no se ha hecho.

Registro de servicios. Los servicios se registran y se consultan por una clave de texto. Las cuatro claves vigentes son `"DataStore"`, `"FileIO"`, `"TaskService"` y `"MainWindow"`.

Brecha: las claves no están centralizadas. Hoy cada clave se escribe como literal en el punto de uso, tanto al registrar en `main.py` como al consultar en cada extensión. Un error de escritura no se detecta: `get_service()` devuelve `None` y el fallo aparece más tarde y lejos de su causa. Lo que corresponde es un módulo con las claves como constantes — por ejemplo `core/services/claves.py` — y que tanto el registro como el consumo las importen de ahí. Nótese además que la clave del servicio de entrada/salida es `"FileIO"` y no `"FileIOService"`, discrepancia que sobrevive precisamente porque no hay un único sitio donde esté escrita.

Catálogo de eventos. El canal es un único `pyqtSignal(str, object)`: un tópico de texto y una carga útil sin tipo. Los tópicos realmente en uso son cinco, y se recogen en la Tabla 4.

Cuadro 4: Eventos publicados en el canal del Kernel

Tópico	Carga útil	Significado
`signal_added`	`{"key": nombre}`	Se cargó una señal nueva.
`signal_active_changed`	nombre de la señal	Cambió la señal activa.
`trials_generated`	`{"signal": ..., çhannel": ...}`	Se recortaron trials de un canal.
`current_trial_changed`	estado del selector	El usuario cambió de trial.
`trial_discard_updated`	descarte aplicado	Se descartó o recuperó un trial.

Brecha respecto al SAD (ADR-03). El SAD prescribe un catálogo con carga útil tipada, de modo que un error de tópico se detecte al emitir. Con el canal genérico actual, publicar `"signal_add"` en lugar de `"signal_added"` no produce ningún error: simplemente no lo escucha nadie. La tabla anterior documenta el catálogo *de facto*; convertirlo en contrato exige constantes para los tópicos y una estructura declarada por carga útil.

Validaciones de carga. El descubridor exige que el `properties.yml` contenga las ocho claves obligatorias (`id`, `name`, `category`, `subcategory`, `version`, `icon`, `logic_class`, `ui_class`) y que la clase declarada en `logic_class` exista dentro de la carpeta. Lo que **no** valida, y el SAD prescribe en ADR-05:

- **Versión del contrato.** El campo `version` del descriptor es la versión de la extensión, no la del contrato `IPlugin` contra el que fue escrita. Una extensión compilada contra un `IPlugin` antiguo se carga igual y falla al usarse.
- **Dependencias.** No existe un campo para declararlas. Una extensión que necesite un servicio no registrado lo descubre en tiempo de ejecución, con un `None`.
- **Unicidad del identificador.** El registro es un diccionario indexado por `id`: dos extensiones con el mismo identificador se sobrescriben sin aviso. El Kernel sí avisa por consola de un *nombre* repetido, pero igualmente lo sobrescribe.

2.4.3. Arranque y cierre

CU-011 — Iniciar la aplicación. El orden de instanciación en `main.py` no es arbitrario; cada paso depende del anterior:

1. **Guarda del temporizador de VTK.** Antes de crear ningún widget se sustituye el manejador de temporizador de `QVTKRenderWindowInteractor` por una versión que comprueba que el widget siga visible. Va primero porque la sustitución tiene que estar puesta antes de que exista el primer interactor.
2. `QApplication`, fuentes, icono y hoja de estilos.
3. **Pantalla de bienvenida**, que informa del avance de los pasos siguientes.
4. **Kernel**.
5. **Servicios del núcleo**: `DataStore`, `FileIO` y `TaskService`. *Van antes del descubrimiento de extensiones*: `register_plugin()` llama a `initialize(kernel)`, y desde ahí una extensión ya puede pedir cualquier servicio. Si estas líneas se movieran después, recibiría `None`.
6. **Descubrimiento e instanciación** de las extensiones con `PluginManager.load_all()`.
7. **Registro en el Kernel** de cada extensión, por el nombre declarado en su descriptor.
8. `MainWindow`, que se construye con el Kernel ya poblado — necesita la lista de extensiones para montar la barra de categorías — y se registra después como servicio, porque las extensiones lo consultan para la barra de estado y los diálogos.
9. **Mostrar la ventana** en el primer ciclo del bucle de eventos.

Inyección de servicios. No hay un contenedor de inyección de dependencias: el patrón es *Service Locator*. Una extensión no recibe sus dependencias en el constructor, sino que las pide al Kernel cuando las necesita, con `kernel.get_service(clave)`. El contrato `IPlugin` ofrece envoltorios para los accesos más frecuentes — `get_datastore()`, `get_active_signal()`, `get_active_trials()`, `get_trials_de_canal()` — que resuelven el servicio y además traducen la ausencia en un aviso al usuario en lugar de en una excepción.

CU-022 — Cerrar la aplicación. El cierre recorre las extensiones registradas y llama al `stop()` de cada una. Qué libera ese `stop()` depende de la extensión; las de análisis que usan el orquestador cancelan además las tareas que tengan en vuelo con `cancel_all_from(id)`, de modo que no quede un hilo trabajador escribiendo sobre una interfaz que ya no existe.

> **Brecha respecto al SAD (ADR-08).** El SAD exige que tras el cierre «no queden hilos, procesos ni contextos gráficos activos». La detención actual no garantiza eso en un caso concreto: una tarea cancelada que no responda en dos segundos se descarta pero no se detiene, porque un hilo de Python no puede terminarse de forma segura desde fuera. El hilo desacoplado sigue vivo hasta que su función termine. Está documentado en el SAD como limitación conocida de ADR-09.

2.4.4. Hilos, tareas y recursos

Reglas de hilos para el desarrollador. Son tres y no admiten excepción:

1. **VTK y Qt solo en el hilo de interfaz.** Ninguna de las dos librerías es segura fuera de él. Un cálculo que toque un `vtkTable` desde un trabajador produce una violación de acceso, no una excepción.

2. **Ninguna extensión crea sus propios hilos.** El cómputo pesado se encola en el orquestador. Antes había extensiones con su propio `QThread`, lo que daba tantos patrones de concurrencia como extensiones y ninguna forma común de cancelar.
3. **El resultado se dibuja en el *slot* de `finished`**, que Qt ejecuta ya en el hilo de interfaz, y siempre tras comprobar que el manejador recibido sea el vigente.

TaskService. Es la implementación del TaskOrchestrator del SAD. Mantiene una cola única y ejecuta **una tarea a la vez** en un hilo trabajador. `submit(fn, owner=...)` devuelve un `TaskHandle` con cuatro señales — `progress`, `finished`, `failed` y `cancelled` — y al cómputo se le inyecta un `TaskContext` con `ctx.cancelled` y `ctx.progress(pct, mensaje)`.

Cada señal se conecta con una guarda. Al cambiar un parámetro se encola otra tarea y la anterior puede seguir viva un instante; sin la guarda `if handle is not self._handle`, su resultado sobrescribiría al nuevo:

```
handle = tasks.submit(cp.ppc_mapa, owner=self.meta.id, ...)
self._handle = handle
handle.progress.connect(partial(self._on_progress, handle))
handle.finished.connect(partial(self._on_done, handle))
handle.failed.connect(partial(self._on_failed, handle))
handle.cancelled.connect(partial(self._on_cancelled, handle))
```

Control de admisión. Existe en una forma mínima: al encolar, el servicio descarta las tareas *pendientes* de la misma extensión (`_discard_queued_from(owner)`), de modo que pulsar *Generate* cinco veces no deja cinco cálculos en cola. No existe, en cambio, la estimación de costo previa que prescribe ADR-10: el sistema no se niega a lanzar un cálculo que vaya a tardar minutos ni advierte de ello.

Cancelación. Es cooperativa: se activa un `threading.Event` y el cómputo lo consulta en sus puntos de comprobación. Si no responde en dos segundos, el servicio deja de esperarlo y descarta su resultado, pero el hilo sigue corriendo.

Política hilo/proceso. El SAD prescribe en ADR-09 un orquestador híbrido con un proceso trabajador para los cómputos aislables. **La implementación usa solo hilos**, y la razón es una medición, no un olvido: el costo de arrancar un proceso y serializar los arreglos hacia él resultó ser de **1.486 ms** frente a los **1.163 ms** que tarda el cálculo en sí. El proceso costaba más que lo que se quería paralelizar. Como las operaciones pesadas son transformadas de NumPy, que liberan el *GIL* mientras calculan, los hilos sí aprovechan varios núcleos: el reparto de trials entre cuatro hilos en PPC rinde entre **1,8**× y **2,7**× con resultado idéntico bit a bit. Lo que los hilos no dan es el aislamiento de fallos ni la cancelación forzada, que siguen siendo el argumento a favor de ADR-09 y quedan pendientes.

Política de memoria. Cuatro mecanismos, en orden de impacto:

- **Submuestreo.** Todos los análisis toman una frecuencia de muestreo objetivo y submuestrean antes de calcular. El factor es entero, así que la tasa pedida no siempre se alcanza: `tasa_efectiva()` devuelve la tasa *real* resultante, y es esa — y no la pedida — la que construye el eje de frecuencias. Tratarlas como iguales corría todo el eje hasta un 11 %.
- **Tipo numérico por etapa.** El cálculo va en `float64` y lo que se entrega a VTK para dibujar se convierte a `float32`, que basta para la pantalla y ocupa la mitad.

- **Cómputo por tandas.** Las transformadas tiempo-frecuencia se calculan en bloques de frecuencias en lugar de reservar la matriz completa: para 500 filas por 60.000 muestras la diferencia es de 480 MB a 30 MB.
- **Memoización de filtros.** El diseño de los filtros IIR está memoizado y devuelve una copia de los coeficientes, para que ningún consumidor pueda alterar la entrada compartida. Al cambiar de trial, el ahorro medido es de **7,25**×.

Liberación determinista. Al salir de una pestaña, `stop()` desactiva el interactor de VTK de la extensión. Está documentado como defecto abierto que la memoria no se libera por completo al cerrar un proyecto.

2.4.5. Fallos, registro y mensajes

Taxonomía de errores. El sistema distingue tres severidades, cada una con su canal, a través de `PluginAlerts`:

Cuadro 5: Severidades y tratamiento

Severidad	Canal	Cuándo se usa
Error	Diálogo modal (`alerts.error`)	El usuario pidió algo que no se puede hacer y tiene que enterarse: un archivo ilegible, un parámetro fuera de rango, dos canales no alineados. Interrumpe.
Aviso	Diálogo o registro (`alerts.warning`)	Se puede continuar, pero hay algo que condiciona la lectura del resultado.
Informativo	Registro y barra de estado	Avance de una tarea, resultado de una operación, condiciones del cálculo. No interrumpe.

Los avisos sobre el resultado de un cálculo no abren diálogo, y hay una razón técnica además de la de usabilidad: un modal abre un bucle de eventos anidado, y el temporizador de `QVTKRenderWindowInteractor` sigue disparando dentro de él sobre widgets que pueden estar ocultos. Tres cierres inesperados de la aplicación se rastrearon hasta ahí. Por eso las extensiones que informan de condiciones del cálculo — el índice de modulación y PPC — lo hacen con una función que devuelve una lista de textos, que el plugin registra. Un fallo de verdad sí abre diálogo, pero ese es otro camino: el *slot* de `failed`.

Captura en los trabajadores. Una excepción dentro de un cómputo no cruza el límite del hilo: el `TaskService` la captura y la convierte en la señal `failed` con su mensaje, que el *slot* correspondiente traduce en un diálogo y en la restauración del botón. El dibujo también va protegido: una excepción al renderizar se registra y se reporta, y el bloque `finally` restituye el estado de la interfaz pase lo que pase.

Archivos corruptos. `FileIOService` valida al leer y lanza una excepción con el motivo; la extensión que la invoca la traduce en un diálogo. Una carpeta de extensión con el descriptor mal formado no impide el arranque: se omite con un mensaje y el resto carga.

> **Brecha respecto al SAD (ADR-11 y ADR-12).** Dos elementos prescritos no existen en el código.

No hay manejador global de excepciones. No se instala ningún `sys.excepthook`. Una excepción no capturada en el hilo de interfaz — fuera de los bloques protegidos — termina el proceso sin que el usuario sepa por qué y sin que quede registro.

No hay registro estructurado. El sistema no usa el módulo `logging`: los mensajes salen por `print()`, a través de `IPlugin._log()`, que antepone el nombre de la extensión. No hay niveles, ni marca de tiempo, ni archivo, ni rotación. Diagnosticar un fallo que le ocurrió al usuario exige reproducirlo, que es exactamente lo que ADR-12 pretende evitar. La migración está acotada — un `logging.getLogger` por módulo y un `RotatingFileHandler` en el arranque — pero toca todas las llamadas existentes a `print`.

2.4.6. Extensibilidad

Contrato `IPlugin`. Toda extensión hereda de `IPlugin` y recibe su descriptor en el constructor. Los métodos que debe implementar son tres — `stop()`, `get_widget(parent)` y `process(data)` — y hereda `initialize(kernel)`, `start(kernel)`, el acceso a los servicios y el manejo de alertas. La Tabla 6 resume el contrato.

Cuadro 6: Contrato `IPlugin`

Método	Origen	Responsabilidad
`initialize(kernel)`	Heredado	Guardar la referencia al Kernel. Se llama al registrar.
`start(kernel)`	Heredado	Obtener `MainWindow`, suscribirse al canal de eventos y marcar `started`.
`stop()`	**Obligatorio**	Liberar lo propio: tareas en vuelo, interactor de VTK, observadores.
`get_widget(parent)`	**Obligatorio**	Construir la vista la primera vez y devolverla siempre.
`process(data)`	**Obligatorio**	Recibir datos del Kernel o de otra extensión.
`get_analysis_params()` / `apply_analysis_params(d)`	Opcional	Guardar y restaurar los parámetros con el proyecto.

Descriptor `properties.yml`. Ocho claves obligatorias: `id` (identificador único), `name` (el que se muestra y con el que se registra), `category` y `subcategory` (dónde aparece el botón; la interfaz agrupa por subcategoría sin lista fija, de modo que una subcategoría nueva aparece sola), `version`, `icon` (archivo dentro de la carpeta), `logic_class` (la clase que implementa `IPlugin`) y `ui_class`. Opcional: `description`, que alimenta la ayuda emergente.

Patrón `compute` / `ui`. Una extensión de análisis se escribe en tres archivos:

- `compute.py` — el cálculo. Recibe NumPy, devuelve NumPy. No importa Qt ni VTK. Es lo que corre en el trabajador y lo que se prueba.
- `<nombre>_plugin_ui.py` — la vista: los controles y el hueco del gráfico.
- `<nombre>_plugin.py` — la lógica de coordinación: lee los parámetros, encola, recibe y dibuja.

La separación no es estética. Permite probar el cálculo sin interfaz — las 76 pruebas del cómputo de PPC corren en segundos y sin Qt — y permite ejecutarlo en un hilo trabajador, que es lo que exige la regla de hilos. Nueve de las diecinueve extensiones siguen hoy este patrón; las de frecuencia (`fft`, `psd` y sus promedios) todavía tienen el cálculo dentro de la clase del plugin.

Puntos de extensión. Hay tres, de distinta madurez:

1. **Extensiones.** Es el punto principal y el único verdaderamente declarativo: basta una carpeta con `properties.yml` bajo `plugins/` para que el sistema la descubra al arrancar. No hay que tocar ningún registro central.
2. **Formatos de archivo y de exportación.** No son declarativos. Un formato de lectura nuevo exige un método en `FileIOService` y una rama en el punto de llamada; uno de exportación, una rama en `ExportService`. Convertirlos en puntos de extensión con registro — un diccionario de extensión de archivo a función lectora — está pendiente.
3. **Eventos.** Cualquier extensión puede publicar un tópico nuevo y cualquiera puede escucharlo, pero sin catálogo ni tipado (véase 2.4.2).

Guía para crear una extensión nueva. Los pasos, en orden:

1. Crear la carpeta bajo la categoría que corresponda, por ejemplo `plugins/analysis/connectivity/<nombr`
2. Escribir `properties.yml` con las ocho claves y colocar el icono.
3. Escribir `compute.py` con el cálculo puro, y sus pruebas en `test/plugins_test/`. Conviene hacerlo antes que la interfaz: es donde están los errores que importan.
4. Escribir `<nombre>_plugin_ui.py` con los controles y el área del gráfico.
5. Escribir `<nombre>_plugin.py`: heredar de `IPlugin`, implementar `get_widget`, `stop` y `process`, encolar el cálculo con `submit()` y conectar las cuatro señales con su guarda.
6. Si el cálculo es compartido con otra extensión, subirlo a `core/filters/` en lugar de duplicarlo.
7. Verificar que la regla de dependencia se mantiene: que `compute.py` no importe Qt ni VTK, y que nada de `core/` importe la extensión nueva.

No hay que modificar `main.py`, ni el Kernel, ni `MainWindow`: el descubrimiento es automático y la agrupación por categoría también.

3. Diseño Detallado

3.1. Objetivo de la sección

Esta sección desarrolla el detalle técnico de GammaLab 2.0: la estructura de clases de cada componente, el comportamiento observable del sistema a nivel de interacción entre objetos, los modelos de datos persistidos en memoria y las relaciones entre entidades. Complementa la Sección 2 (Arquitectura), que describe el sistema a nivel de componentes y procesos, con el nivel de detalle necesario para la implementación.

3.2. Estructura del Sistema

3.2.1. Propósito

El propósito de esta sección es documentar la estructura de clases de cada componente del sistema, sirviendo como guía para su implementación.

3.2.2. Organización general

El código fuente de GammaLab 2.0 sigue la misma organización por responsabilidades presentada en la Vista Lógica (Sección 2.1): cada componente concentra un conjunto cohesivo de clases y se comunica con los demás mediante los servicios registrados en el `Kernel`. El diagrama de clases completo del sistema se presenta en la Figura 1 (Sección 2.1); a continuación se detalla cada componente por separado.

3.2.3. Descripción de clases por componentes

1. **Componente: ApplicationCore**

   El componente **ApplicationCore** concentra el núcleo del sistema: el registro de plugins y servicios, el contrato de extensión (`IPlugin`) y las utilidades compartidas por el resto de la aplicación.

   Las clases del componente *ApplicationCore* son las siguientes:

   PluginMeta Metadatos y configuración de un plugin descubierto por el sistema.

   - **Atributos principales:** `id`, `name`, `category`, `subcategory`, `version`, `icon`: `str`; `logic_class`, `ui_class: Optional[str] = None`; `root: Optional[Path] = None`.
   - **Métodos principales:** `icon_path(): Path`.

   PluginManager Gestor central de descubrimiento y registro de plugins a partir de sus archivos `properties.yml`.

   - **Atributos principales:** `plugins_dir: Path`; `registry: Dict[str, Tuple[PluginMeta, IPlugin]]`.
   - **Métodos principales:** `load_all()`, `all()`, `get(plugin_id)`.

   IPlugin («interface») Contrato que toda extensión del sistema implementa.

   - **Atributos principales:** `meta: PluginMeta`; `started: bool`; `mainwin`; `widget: QWidget`; `kernel: Kernel`; `active_signal: SignalDataset`; `active_chanel`; `alerts: PluginAlerts`.
   - **Métodos principales:** `name()`/`category()`/`subcategory()`, `icon()`, `get_datastore()`, `get_active_signal()`, `get_active_trials(signal)`, `on_kernel_event(topic, payload)`, `initialize(kernel)`, `start(kernel)`, `stop()` {abstracto}, `get_widget(parent=None)` {abstracto}, `process(data)` {abstracto}.

   Kernel Núcleo central de la aplicación; implementa el patrón microkernel con inversión de control sobre servicios y plugins.

   - **Atributos principales:** `event: pyqtSignal(str, object)`; `plugin_registered: pyqtSignal(str)`; `_plugins: dict`; `_services: dict`.

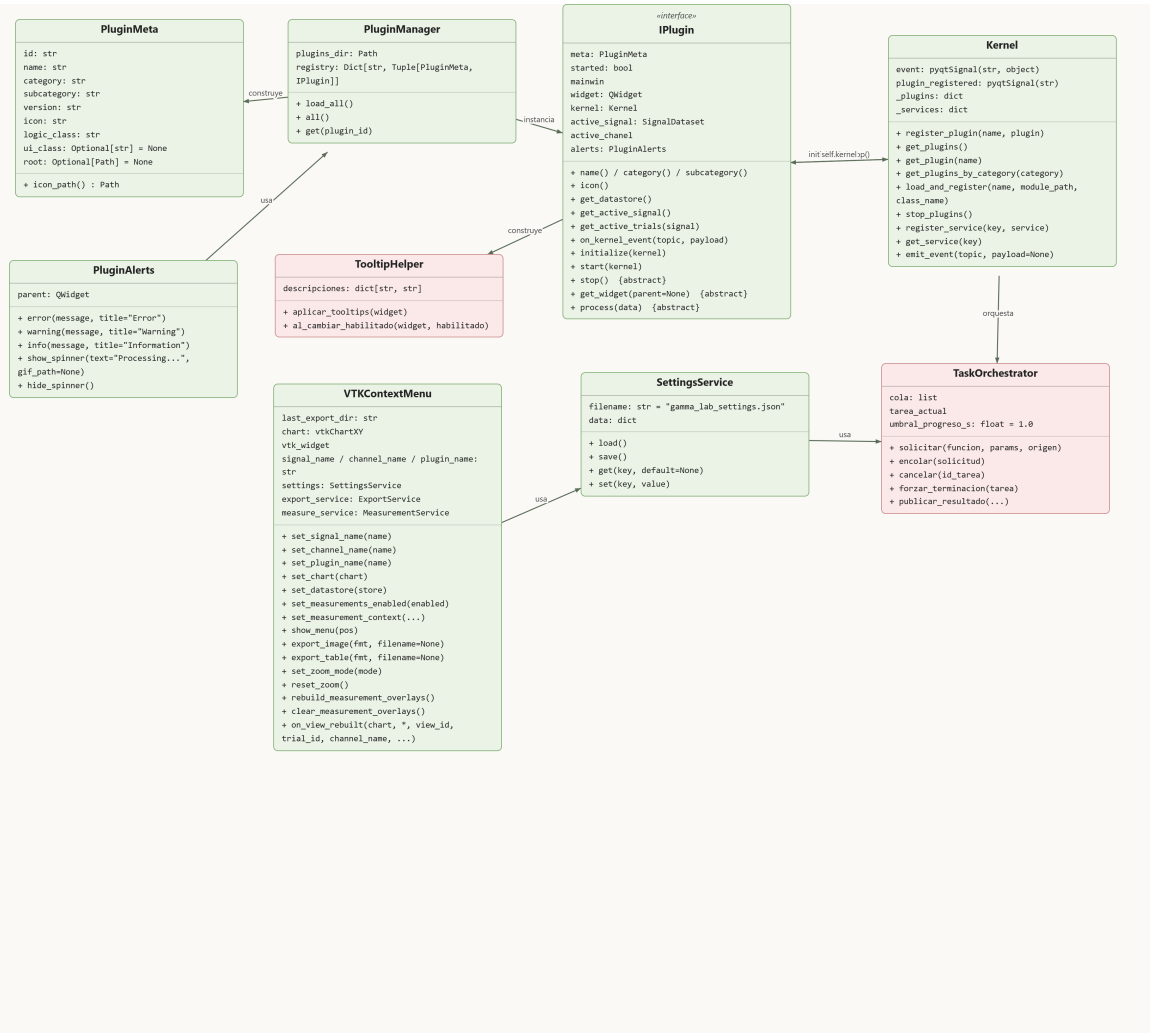


Figura 4: Diagrama de clases — componente ApplicationCore

- **Métodos principales:** `register_plugin(name, plugin)`, `get_plugins()`, `get_plugin(name)`, `get_plugins_by_category(category)`, `load_and_register(name, module_path, class_name)`, `stop_plugins()`, `register_service(key, service)`, `get_service(key)`, `emit_event(topic, payload=None)`.

SettingsService Servicio de configuración general del sistema y preferencias del usuario.

- **Atributos principales:** `filename: str = "gamma_lab_settings.json"`; `data: dict`.
- **Métodos principales:** `load()`, `save()`, `get(key, default=None)`, `set(key, value)`.

VTKContextMenu Menú contextual general para vistas basadas en VTK; delega en `ExportService` y `MeasurementService` (ver esos componentes).

- **Atributos principales:** `last_export_dir: str`; `chart: vtkChartXY`; `vtk_widget`; `signal_name/channel_name/plugin_name: str`; `settings: SettingsService`; `export_service: ExportService`; `measure_service: MeasurementService`.
- **Métodos principales:** `set_signal_name(name)`, `set_channel_name(name)`, `set_plugin_name(name)`, `set_chart(chart)`, `set_datastore(store)`, `set_measurements_enabled(enabled)`, `set_measurement_context(...)`, `show_menu(pos)`, `export_image(fmt, filename=None)`, `export_table(fmt, filename=None)`, `set_zoom_mode(mode)`, `reset_zoom()`, `rebuild_measurement_overlays()`, `clear_measurement_overlays()`, `on_view_rebuilt(chart, *, view_id, trial_id, channel_name, ...)`.

PluginAlerts Sistema de notificaciones y alertas reutilizado por los servicios y los plugins.

- **Atributos principales:** `parent: QWidget`.
- **Métodos principales:** `error(message, title=."Error")`, `warning(message, title="Warning")`, `info(message, title="Ïnformation")`, `show_spinner(text="Processing...", gif_path=None)`, `hide_spinner()`.

TaskOrchestrator Centraliza la ejecución de cómputo pesado (PAC, Wavelet, PSD) fuera del hilo de interfaz, con soporte de cancelación.

- **Atributos principales:** `cola: list`; `tarea_actual`; `umbral_progreso_s: float = 1.0`.
- **Métodos principales:** `solicitar(funcion, params, origen)`, `encolar(solicitud)`, `cancelar(id_tarea)`, `forzar_terminacion(tarea)`, `publicar_resultado(...)`.

TooltipHelper Ayuda contextual mediante tooltips dinámicos sobre los controles de la interfaz.

- **Atributos principales:** `descripciones: dict[str, str]`.
- **Métodos principales:** `aplicar_tooltips(widget)`, `al_cambiar_habilitado(widget, habilitado)`.

`TaskOrchestrator` y `TooltipHelper` son clases nuevas: no existen aún en el código de GammaLab 2.0. Las demás clases del componente sí están implementadas, con una salvedad: `SettingsService` está completa pero no se registra en `main.py` hoy, por lo que ningún plugin la consume actualmente a través del Kernel.

2. **Componente: MainUI**

El componente **MainUI** provee al núcleo la interfaz principal de la aplicación.

MainWindow Ventana principal de la aplicación; se registra a sí misma como servicio del Kernel ("MainWindow") y coordina la interfaz con los plugins.

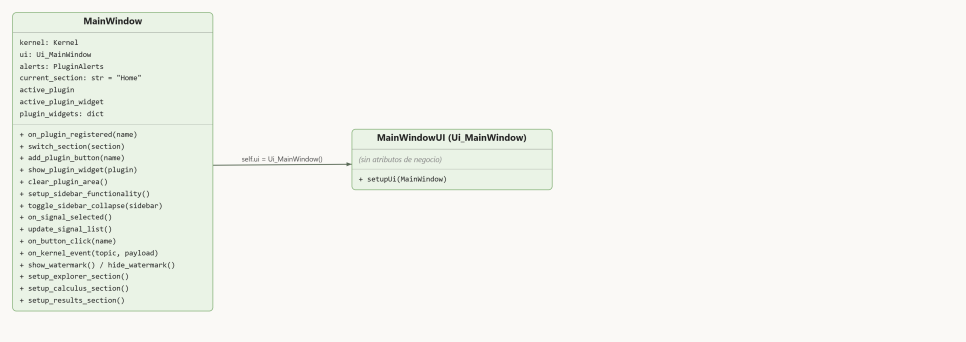


Figura 5: Diagrama de clases — componente MainUI

- **Atributos principales:** `kernel: Kernel`; `ui: Ui_MainWindow`; `alerts: PluginAlerts`; `current_section: str = "Home"`; `active_plugin`; `active_plugin_widget`; `plugin_widgets: dict`.
- **Métodos principales:** `on_plugin_registered(name)`, `switch_section(section)`, `add_plugin_button(name)`, `show_plugin_widget(plugin)`, `clear_plugin_area()`, `setup_sidebar_functionality()`, `toggle_sidebar_collapse(sidebar)`, `on_signal_selected()`, `update_signal_list()`, `on_button_click(name)`, `on_kernel_event(topic, payload)`, `show_watermark()`/`hide_watermark()`, `setup_explorer_section()`, `setup_calculus_section()`, `setup_results_section()`.

MainWindowUI (`Ui_MainWindow`) Construcción visual de la ventana principal, sin lógica de negocio propia.

- **Métodos principales:** `setupUi(MainWindow)`.

`MainWindow` consulta el servicio `DataStore` (vía `kernel.get_service("DataStore")`) para poblar la lista de señales, e invoca `get_widget()`, `start()`, `process()` y `stop()` de cada `IPlugin` registrado.

3. **Componente: Plugins**

   El componente **Plugins** es el punto de extensión del sistema: toda funcionalidad de entrada/salida, preprocesamiento y análisis se implementa como un plugin que respeta el contrato `IPlugin`.

   OpenSignalPlugin, **TrialsPlugin**, **Filter_plugin** y **Wavelet_plugin** son cuatro ejemplos representativos de los plugins ya existentes en el sistema (el repositorio incluye 16 en total: FFT, PSD, Average, ERP, Amplitude, Slope, ArtifactRemove, FAQ, entre otros). Todos implementan `get_widget()` y `process(data)` de `IPlugin`, y son descubiertos por `PluginManager` a partir de su `properties.yml`; el nombre de la clase declarado ahí (`logic_class`) no siempre coincide con el nombre de la carpeta del plugin.

   PACAnalysisPlugin («abstract», nuevo) Base común de los plugins de acoplamiento fase-amplitud (*Phase-Amplitude Coupling*).

   - **Atributos principales:** `banda_fase: tuple`; `rango_amplitud: tuple`.
   - **Métodos principales:** `calcular_mapa_fase_frecuencia()`.

   PACSingleChannelPlugin (nuevo) Hereda de `PACAnalysisPlugin`; calcula el PAC de un único canal.

   - **Métodos principales:** `calcular_pac_1_canal()`.

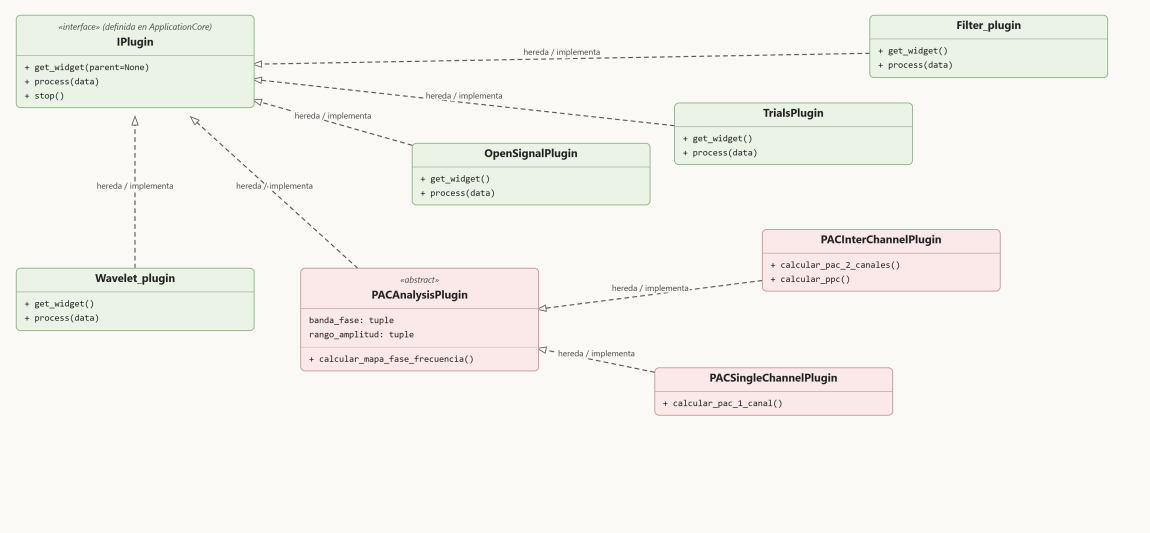


Figura 6: Diagrama de clases — componente Plugins

PACInterChannelPlugin (nuevo) Hereda de `PACAnalysisPlugin`; calcula el PAC cruzado entre dos canales.

- **Métodos principales:** `calcular_pac_2_canales()`, `calcular_ppc()`.

El cómputo pesado de los plugins de análisis (Wavelet, familia PAC) se despacharía al `TaskOrchestrator` (ver componente ApplicationCore) en vez de bloquear el hilo de interfaz.

4. **Componente: DataStore**

   El componente **DataStore**, encargado de la persistencia en memoria, provee un servicio de almacenamiento de tipo clave-valor y define los modelos de datos centrales del sistema.



Figura 7: Diagrama de clases — componente DataStore

DataStore Almacén centralizado en memoria de señales, trials y cualquier otro estado indexado por clave.

- **Atributos principales:** `_data: dict`.
- **Métodos principales:** `set(key, value)`, `get(key, default=None)`, `has(key)`, `items()`, `remove(key)`, `add_signal(signal, key=None)`, `get_signals()`, `set_active_signal(key)`, `get_active_signal()`, `get_active_signal_key()`, `clear_active_signal()`, `is_active_signal(key)`.

SignalDataset Conjunto completo de datos de una señal cargada, incluyendo sus trials asociados.

- **Atributos principales:** `name: str`; `format: str`; `source_path: str`; `sampling_rate: float`; `time: np.ndarray`; `signals: np.ndarray`; `channel_names: List[str]`; `units: List[str]`; `metadata: Dict[str, Any]`; `vtk_table = None`; `__trials_dataset: List[TrialDataset]`; `__discarded_trials: Dict[tuple[str,str], set[int]]`.
- **Métodos principales:** `add_trial_dataset(trial)`, `number_of_trials_dataset()`, `get_active_trials(file_name, channel_name=None)`, `discard_trial(source, channel, index)`, `include_trial(source, channel, index)`, `clear_discarded_trials()`, `is_trial_discarded(source, channel, index)`.

TrialDataset Partición (ensayo) de una señal, usada para análisis focalizados.

- **Atributos principales:** `source: str`; `sampling_rate: float`; `channel_index: int`; `channel_name: str`; `unit: str`; `t0`/`t1: float`; `time_rel: np.ndarray`; `trials: np.ndarray (Ns, T)`; `onsets_s: List[float]`; `isi_s: List[float]`; `metadata: Dict[str, Any]`.
- **Métodos principales:** sin métodos propios — es una estructura de datos consumida por `SignalDataset`.

`FileIOService` (ver componente FileIO) construye instancias de `SignalDataset` a partir de archivos externos; `TaskOrchestrator` y `ProjectSession` (ver esos componentes) leerían y escribirían en `DataStore`.

5. **Componente: FileIO**

El componente **FileIO** es responsable de la lectura de archivos de señal EEG en múltiples formatos.

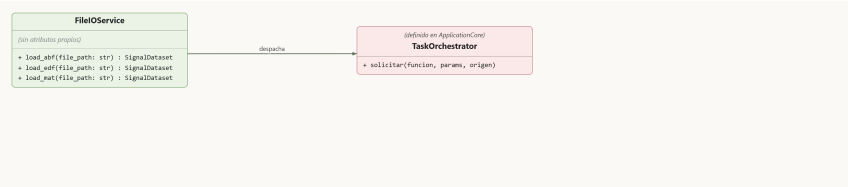


Figura 8: Diagrama de clases — componente FileIO

FileIOService Servicio de lectura de archivos que construye instancias de `SignalDataset`.

- **Métodos principales:** `load_abf(file_path: str): SignalDataset`, `load_edf(file_path: str): SignalDataset`, `load_mat(file_path: str): SignalDataset`.

`FileIOService` usa las librerías externas `pyabf`, `pyedflib`, `scipy.io.loadmat` y `h5py`. El Kernel registra el servicio bajo la clave `"FileIO"` (no `"FileIOService"`); si `OpenSignalPlugin` no lo encuentra registrado, crea uno propio y lo registra él mismo. `load_mat()` está implementado y funciona correctamente, pero la rama que lo invoca en `OpenSignalPlugin` está comentada hoy: desde la interfaz solo se pueden cargar archivos `.abf` y `.edf`. El SAD (Escenario de Calidad 2) documenta que esta lectura debería despacharse al `TaskOrchestrator` para no bloquear la interfaz; hoy `OpenSignalPlugin` llama a `load_abf()`/`load_edf()` de forma directa y síncrona en el hilo de interfaz, sin que el orquestador intervenga todavía en esta ruta.

6. **Componente: Export**

El componente **Export** es responsable de la generación de archivos de salida (imagen y tabla) a partir de las visualizaciones activas.

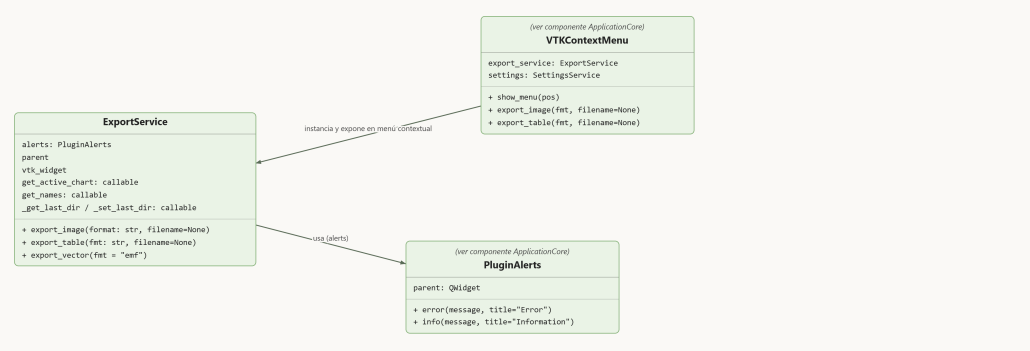


Figura 9: Diagrama de clases — componente Export

ExportService Servicio de exportación de imágenes y tablas a partir de la visualización VTK activa.

- **Atributos principales:** `alerts: PluginAlerts`; `parent`; `vtk_widget`; `get_active_chart/get_names: callable`; `_get_last_dir/_set_last_dir: callable`.
- **Métodos principales:** `export_image(format: str, filename=None)`, `export_table(fmt: str, filename=None)`, `export_vector(fmt = ."emf")`.

`ExportService` no depende directamente de `VTKContextMenu`: recibe callbacks para desacoplarse de la interfaz que lo construye. Aun así, `VTKContextMenu` (ver componente ApplicationCore) sí lo instancia en su constructor y expone sus métodos en el menú contextual de cada gráfico; `ExportService` crea además su propia instancia de `PluginAlerts`. El SAD (Escenario de Calidad 4) describe a `ExportService` como registrado en el núcleo por inversión de control, pero `main.py` solo registra los servicios `"DataStore"` y `"FileIO"`: `ExportService` no está registrado en el Kernel, solo existe la instancia que crea `VTKContextMenu`. `export_vector()` corresponde al caso de uso CU-017; no existe implementación de exportación vectorial (`.emf`) en el código — solo están implementados `export_image()` (png/jpg/jpeg/bmp/tiff) y `export_table()` (csv/xlsx/json).

7. **Componente: Measurement**

El componente **Measurement** contiene la lógica para realizar mediciones interactivas de pendiente y amplitud sobre las señales.

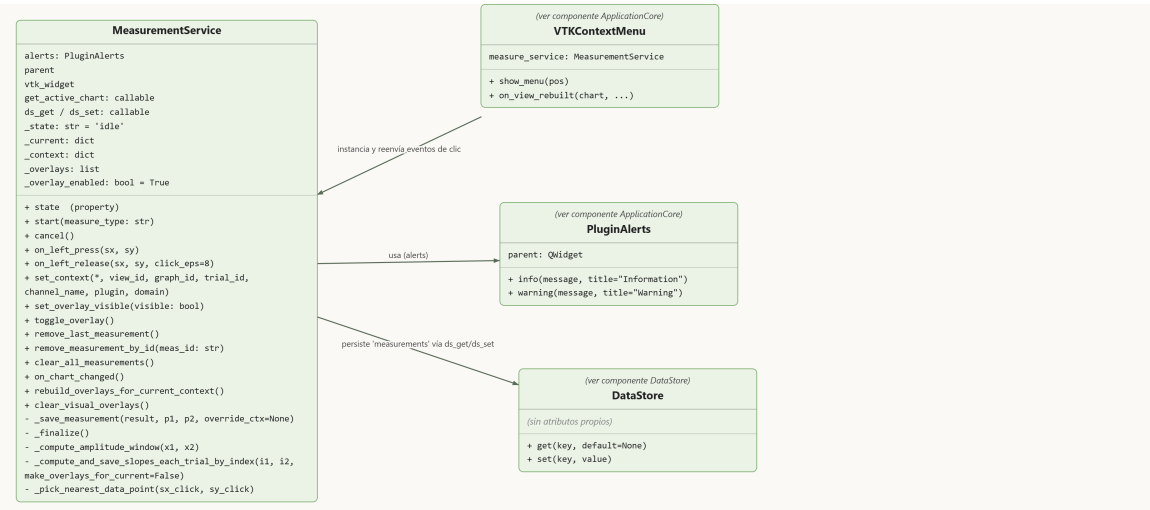


Figura 10: Diagrama de clases — componente Measurement

MeasurementService Servicio de medición que gestiona la selección de puntos, el cálculo y la persistencia de los resultados.

- **Atributos principales:** `alerts: PluginAlerts`; `parent`; `vtk_widget`; `get_active_chart: callable`; `ds_get/ds_set: callable`; `_state: str = 'idle'`; `_current: dict`; `_context: dict`; `_overlays: list`; `_overlay_enabled: bool = True`.
- **Métodos principales:** `state` (propiedad), `start(measure_type: str)`, `cancel()`, `on_left_press(sx, sy)`, `on_left_release(sx, sy, click_eps=8)`, `set_context(...)`, `set_overlay_visible(visible: bool)`, `toggle_overlay()`, `remove_last_measurement()`, `remove_measurement_by_id(meas_id: str)`, `clear_all_measurements()`, `on_chart_changed()`, `rebuild_overlays_for_current_context()`, `clear_visual_overlays()`.

`VTKContextMenu` (ver componente ApplicationCore) instancia `MeasurementService` en su constructor y le reenvía los eventos de clic izquierdo del `vtk_widget`. Los resultados se persisten en `DataStore` bajo la clave `"measurements"` — la misma instancia que ya almacena señales y trials — mediante los callbacks `ds_get/ds_set`. `MeasurementService` crea su propia instancia de `PluginAlerts` y usa `two_point_metrics()/amplitude_in_window()` de `core/filters/measurements.py` para el cálculo numérico. Al igual que `ExportService`, no está registrado como servicio del Kernel pese a que el SAD lo describe así: solo existe la instancia que crea `VTKContextMenu`.

8. **Componente: ProjectSession** (nuevo, sin equivalente en GammaLab 1.0)

   El componente **ProjectSession** materializa el atributo de calidad "Trazabilidad y Persistencia"que GammaLab 2.0 introduce: la persistencia local del espacio de trabajo completo en archivos de proyecto `.glab`.

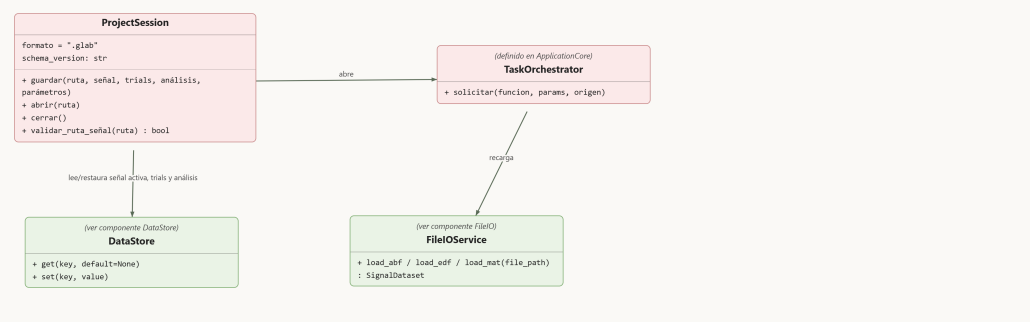


Figura 11: Diagrama de clases — componente ProjectSession

ProjectSession (nuevo) Gestiona la persistencia del espacio de trabajo en archivos `.glab`.

- **Atributos principales:** `formato = ".glab"`; `schema_version: str`.
- **Métodos principales:** `guardar(ruta, señal, trials, análisis, parámetros)`, `abrir(ruta)`, `cerrar()`, `validar_ruta_señal(ruta): bool`.

Ninguna clase de este componente existe hoy en el código — hay cero referencias a `".glab"` en todo el repositorio (SAD, UC-03) — pero sus relaciones sí se apoyan en clases reales: `guardar()` persiste señal, trials, análisis y parámetros en el `.glab` y libera la memoria de la señal activa; `abrir(ruta)` verifica con `validar_ruta_señal()` que el archivo de señal referenciado siga disponible y despacha su recarga al `TaskOrchestrator` fuera del hilo de interfaz, que a su vez reutilizaría `FileIOService` (ver componente FileIO); los trials y análisis se restauran directamente en `DataStore` mediante escritura sincronizada. Cada `.glab` embebe su propio `schema_version`: un incremento MINOR se mantiene legible por versiones futuras de `ProjectSession`, y un incremento MAJOR rechaza la apertura con un mensaje explícito en vez de migrar implícitamente.

3.2.4. Resumen

- **Patrón arquitectónico:** inversión de control mediante el `Kernel`, que expone `register_service()`/`get_service()` y desacopla a los plugins de la construcción directa de sus dependencias.
- **Relación central:** todos los plugins concretos implementan `IPlugin` y son descubiertos y gestionados por `PluginManager`/`Kernel` a partir de su `properties.yml`.
- **Servicios resueltos vía Kernel:** `DataStore` y `FileIOService` (bajo la clave `"FileIO"`) sí se consumen mediante `kernel.get_service()`.
- **Servicios instanciados directamente:** `ExportService` y `MeasurementService` son construidos directamente por `VTKContextMenu`, y `SettingsService` no se registra en `main.py`; los tres aparecen en el SAD como servicios del núcleo, pero ninguno se resuelve hoy mediante `kernel.get_service()`.
- **Piezas construidas desde la redacción inicial de esta sección:** el `TaskOrchestrator` existe como `core/services/task_service.py`, se registra en `main.py` bajo la clave `"TaskService"` y lo consumen siete extensiones; la persistencia de proyecto existe como `core/services/project_service.` y la familia de análisis de PAC existe como `Pac_plugin` con sus tres modos, además de `Mi_plugin` y `Ppc_plugin`. Su funcionamiento se documenta en la Sección 2.4.
- **Piezas todavía prospectivas:** `TooltipHelper`. Y de las decisiones del SAD siguen sin implementarse la máquina de estados del ciclo de vida (ADR-02), el catálogo tipado de eventos (ADR-03), el control de admisión por costo (ADR-10), el manejador global de excepciones (ADR-11) y el registro con archivo rotativo (ADR-12); la Tabla 3 recoge el estado de las trece.
- **Flujo de control:** `MainWindow` actúa como contenedor principal que orquesta la interacción entre plugins a través del `Kernel`.

3.3. Comportamiento del Sistema

3.3.1. Propósito

El objetivo de esta sección es documentar los flujos más importantes del sistema a nivel de interacción entre objetos, complementando la Vista de Procesos (Sección 2.3), que describe el mismo comportamiento a nivel de actividades del usuario.

3.3.2. Principales flujos del sistema

Los flujos descritos abarcan cuatro funcionalidades: la carga de una señal, la toma de mediciones, la exportación de resultados y — como flujo prospectivo, aún no implementado — el guardado y reapertura de un proyecto.

1. Cargar una señal. Desde la pestaña *Home*, el usuario activa *Load Signal*; `OpenSignalPlugin` obtiene el servicio `FileIOService` del Kernel (clave `"FileIO"`) o crea una instancia propia si no lo encuentra registrado. Según la extensión del archivo, invoca `load_abf()` o `load_edf()` de forma directa y síncrona sobre el hilo de interfaz — la rama de `load_mat()` existe en `FileIOService` pero está comentada en el plugin. El resultado, una instancia de `SignalDataset`, se publica en `DataStore` como señal activa y `MainWindow` actualiza la lista de señales disponibles.

2. Hacer mediciones. El usuario hace clic derecho sobre un gráfico, lo que activa `VTKContextMenu.show_menu()` y despliega el menú contextual con las opciones *Slope*, *Amplitude* y *Slope (all trials)*. Al seleccionar una, se invoca `MeasurementService.start(measure_type)`, que usa `PluginAlerts` para solicitar al usuario los dos puntos requeridos. Cada clic del usuario se recibe mediante `on_left_press()`/`on_left_release()`; una vez capturados ambos puntos, `_finalize()` calcula el resultado con `two_point_metrics()` (pendiente) o `amplitude_in_window()` (amplitud) de `core/filters/measurements.py`, dibuja el overlay correspondiente y persiste el registro en `DataStore` bajo la clave `"measurements"` mediante `_save_measurement()`.

3. Exportar resultados. El usuario hace clic derecho sobre un gráfico y selecciona *Export as image* o *Export table* desde el menú de `VTKContextMenu`. Según la opción, se invoca `ExportService.export_image(format)` — que captura la ventana VTK con `vtkWindowToImageFilter` y la escribe con el *writer* correspondiente a png/jpg/jpeg/bmp/tiff — o `export_table(fmt)`, que recorre los *plots* del gráfico activo y escribe el resultado en csv, xlsx o json. En ambos casos, `ExportService` usa `PluginAlerts` para confirmar el éxito o reportar un error, sin interrumpir el estado del sistema.

4. Guardar y reabrir un proyecto. *(Flujo descrito como prospectivo en la redacción inicial. Existe hoy `core/services/project_service.py`, y las extensiones de análisis implementan `get_analysis_params()` / `apply_analysis_params()` para guardar y restaurar sus parámetros; el párrafo siguiente describe el diseño previsto en el SAD y debe contrastarse con esa implementación.)* Al cerrar el proyecto activo, el investigador elige *Guardar*; `ProjectSession.guardar()` persistiría la señal activa, los trials, los análisis y los parámetros de la sesión en un archivo `.glab`, liberando a continuación la memoria de la señal activa. Al abrir un `.glab` existente, `abrir(ruta)` verificaría con `validar_ruta_señal()` que el archivo de señal referenciado siga disponible, despacharía su recarga al `TaskOrchestrator` — que reutilizaría `FileIOService` fuera del hilo de interfaz — y restauraría los trials y análisis directamente en `DataStore` mediante escritura sincronizada. Ninguna clase de este flujo existe hoy en el código: es el criterio de aceptación para la funcionalidad descrita en el SAD (UC-03).

3.4. Persistencia y Modelos de Datos

3.4.1. Propósito

El objetivo de esta sección es describir las estructuras en las que se almacenan los datos de la aplicación y los mecanismos de persistencia que permiten el funcionamiento de los plugins. El sistema no emplea bases de datos: la persistencia es temporal y está basada en un almacén clave-valor en memoria (`DataStore`), accedido a través de sus métodos `get`/`set`.

3.4.2. Entidades de datos

El diagrama de clases del componente DataStore, ya presentado en la Figura 7 (Sección 3.2.3), ilustra las tres entidades centrales del modelo de datos: `DataStore`, `SignalDataset` y `TrialDataset`.

DataStore Servicio central de almacenamiento; mantiene un diccionario interno (`_data`) indexado por clave, sin imponer un esquema fijo. Sus usos principales son:

- Ciclo de vida de las señales cargadas: `add_signal()`, `get_signals()`, `set_active_signal()`/`get_active_signal()`.
- Almacenamiento genérico de cualquier otro estado bajo una clave arbitraria mediante `set(key, value)`/`get(key, default)`: además de señales, `MeasurementService` (ver componente Measurement) lo reutiliza para persistir las mediciones bajo la clave `"measurements"`, sin que `DataStore` distinga entre ambos tipos de contenido.

SignalDataset Representa una señal completa cargada por `FileIOService` (ver componente FileIO): incluye los arreglos de tiempo y de señal, los metadatos de canal y muestreo, y la lista de `TrialDataset` asociados a ella.

TrialDataset Representa un ensayo (segmento) de una señal, con su propia base de tiempo relativa, sus marcadores de *onset* e *ISI*, y sus metadatos de canal y fuente. No tiene métodos propios: es una estructura de datos consumida por `SignalDataset`.

3.5. Relaciones

La relación principal del modelo de datos es la que existe entre `SignalDataset` y `TrialDataset`: la multiplicidad 1 a muchos (1..*) indica que una señal puede contener múltiples trials, cada uno representando un segmento de interés para el análisis. Esta relación se mantiene igual que en GammaLab 1.0.

Sobre esa base, GammaLab 2.0 generaliza el uso de `DataStore` más allá de señales y trials: al ser un almacén clave-valor sin esquema fijo, la misma instancia registrada en el Kernel también persiste las mediciones de `MeasurementService` (clave `"measurements"`), y sería el punto de lectura/escritura de `TaskOrchestrator` y `ProjectSession` una vez implementados.

Un segundo patrón de relación distingue a los componentes según cómo se resuelven sus dependencias: `DataStore` y `FileIOService` se consumen mediante `kernel.get_service()`, mientras que `ExportService` y `MeasurementService` se obtienen por inyección directa desde `VTKContextMenu` en su constructor, sin pasar por el Kernel. Ambos patrones conviven hoy en el sistema, aunque el SAD describe a los cinco servicios como igualmente registrados en el núcleo.

3.6. Interfaz de Usuario

3.6.1. Proposito

Esta sección describe la interfaz de usuario de Gamma Lab, incluyendo la organización, navegación, y componentes de las vistas.

3.6.2. Organización de la pantalla principal

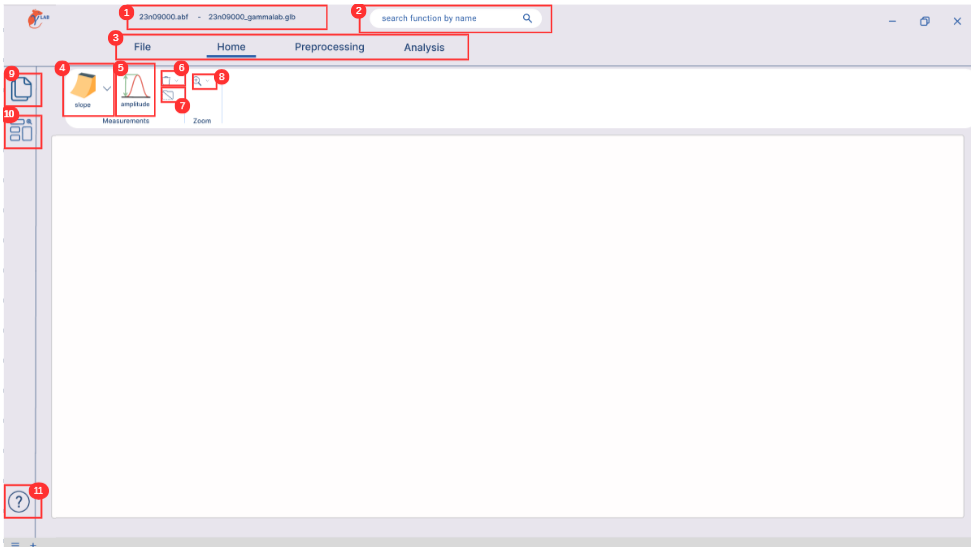


Figura 12: Organización de la pantalla principal de Gamma Lab

La pantalla principal se compone de los siguientes elementos, numerados en la Figura 12:

- **1 - Panel de control:** muestra el nombre de la señal abierta junto al nombre, editable, del proyecto activo, guardado con extensión `.glab`.
- **2 - Barra de búsqueda:** ubicada en la parte superior, permite localizar y acceder rápidamente a una funcionalidad específica del sistema sin tener que recorrer manualmente la barra de navegación.
- **3 - Barra de navegación:** agrupa las pestañas *File*, *Home*, *Preprocessing* y *Analysis*, permitiendo alternar entre la gestión de archivos, las mediciones sobre la señal, el preprocesamiento y los módulos de análisis.
- **4 - slope:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la pendiente de la señal entre dos puntos seleccionados por el usuario, mediante las subopciones *Slope 2 Trials* (calculada sobre dos ensayos) y *Slope All Trials* (calculada sobre la totalidad de los ensayos generados).

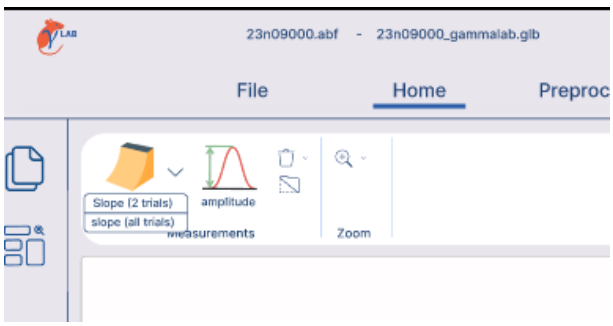


Figura 13: opciones de slope

- **5 - amplitude:** opción de la categoría *Measurements* (pestaña *Home*) que permite medir la amplitud de la señal entre dos puntos seleccionados por el usuario sobre la gráfica.

- **6 - eliminar medida:** permite eliminar la última medición de *slope* o *amplitude* realizada, o bien la totalidad de las mediciones registradas sobre la señal.



Figura 14: opciones de eliminar medida

- **7 - mostrar/ocultar:** permite mostrar u ocultar en pantalla las mediciones de *slope* y *amplitude* realizadas previamente sobre la señal, sin necesidad de eliminarlas.
- **8 - zoom:** activa o desactiva la función de acercamiento sobre la gráfica de la señal, permitiendo observar con mayor detalle un segmento de la señal y las mediciones realizadas sobre él.



Figura 15: opciones de zoom

- **9 - explorador de archivos:** despliega, desde la barra lateral izquierda, la carpeta del sistema de archivos en la que se encuentra almacenado el proyecto actual junto con las señales y resultados exportados, sin salir de la aplicación.

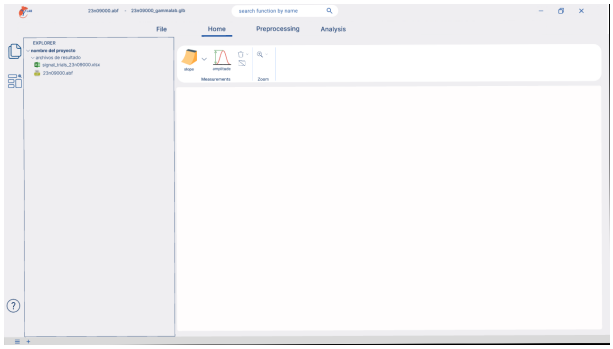


Figura 16: explorador de archivos desplegado

- **10 - resultados:** despliega, desde la barra lateral izquierda, una ventana inferior con los resultados numéricos de las mediciones de *slope* y *amplitude* realizadas sobre la señal, los cuales pueden exportarse en formato `.csv` para su análisis externo.

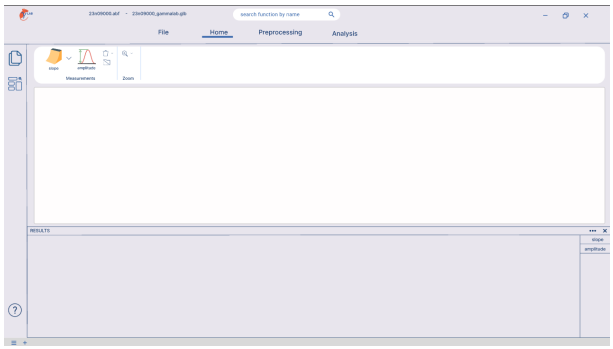


Figura 17: resultados desplegados

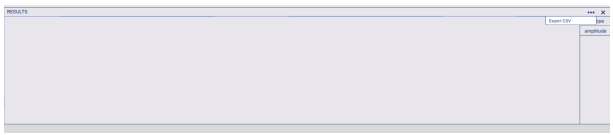


Figura 18: Los resultados pueden ser exportados en formato csv.

- **11 - ayuda:** despliega, desde la barra lateral izquierda, el manual de usuario del sistema, con información de referencia sobre el uso de cada funcionalidad.

3.6.3. Carga y guardado de archivos (pestaña *File*)

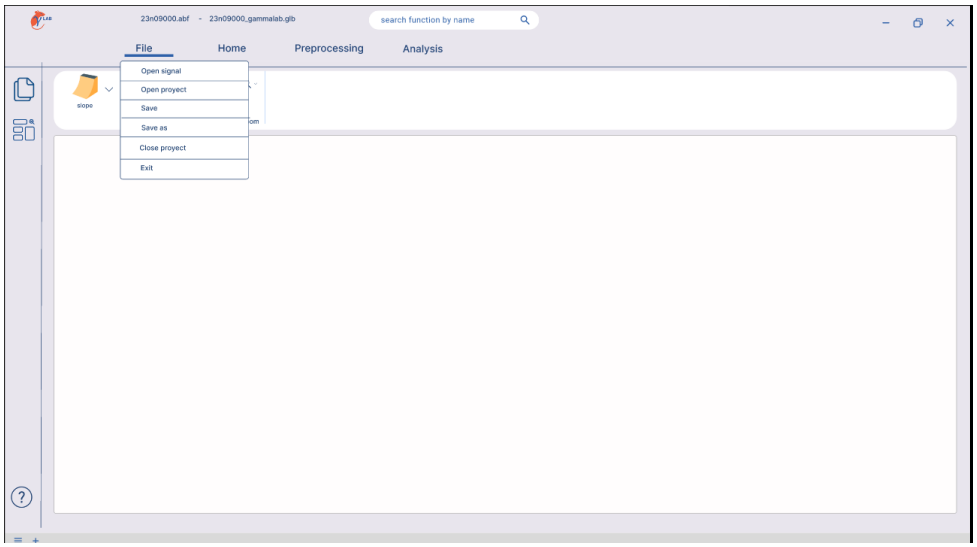


Figura 19: Pestaña File de Gamma Lab

La pestaña *File* permite gestionar la carga y el guardado de señales y proyectos. Agrupa las siguientes opciones:

- **Open Signal:** permite cargar una señal individual dentro de la aplicación para su preprocesamiento y análisis.

- **Open Project:** permite abrir un proyecto previamente guardado con extensión `.glab`, restaurando la señal activa, los trials, los análisis y los parámetros de la sesión.
- **Save:** guarda los cambios realizados sobre el proyecto actualmente abierto.
- **Save As:** permite guardar el proyecto actual bajo un nuevo nombre o ubicación, sin sobrescribir el archivo original.
- **Close Project:** cierra el proyecto que se encuentra abierto, liberando la señal activa de memoria.
- **Exit:** cierra la aplicación.

A continuación se ilustra el flujo de carga de una señal y de un proyecto:

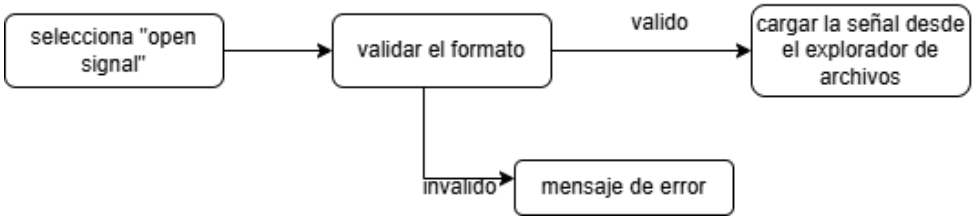


Figura 20: Flujo de carga de la señal (*Open Signal*)

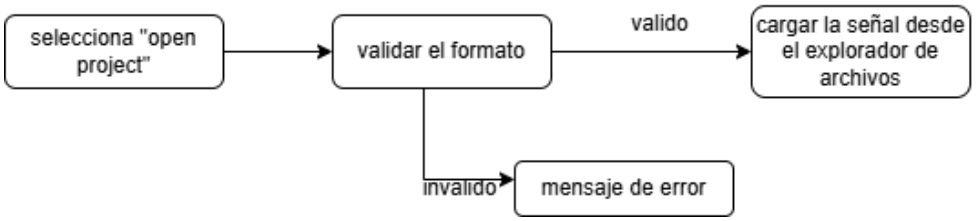


Figura 21: Flujo de carga de proyecto (*Open Project*)

Mediante las opciones *Save* y *Save As* es posible almacenar el estado completo del espacio de trabajo, incluyendo la señal activa, los trials configurados, las medidas realizadas y los resultados de los análisis ejecutados, en un archivo de proyecto con extensión `.glab`.

3.6.4. Preprocesamiento

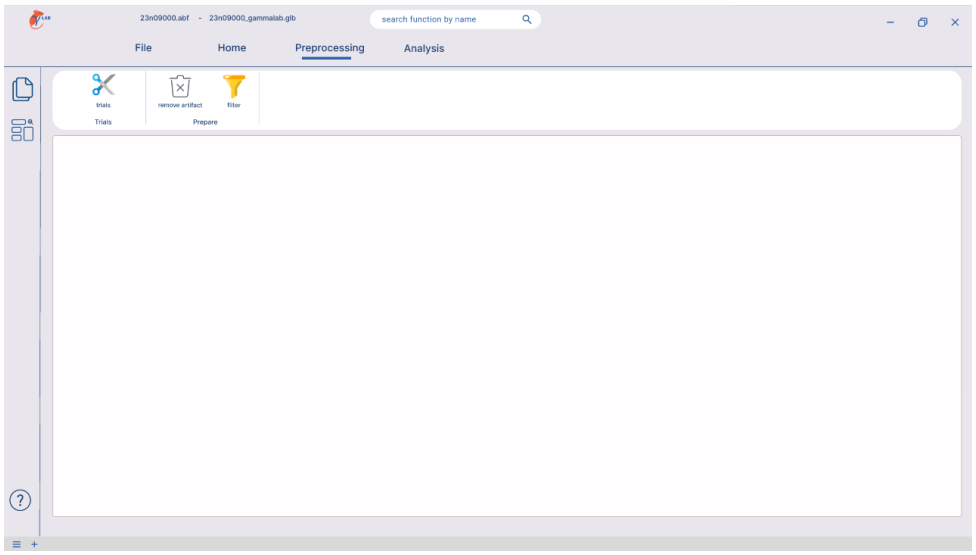


Figura 22: Pestaña Preprocessing de Gamma Lab

Categoria *Trials*

- **trials:** divide la señal cargada en ensayos (*trials*), permitiendo seleccionar el canal a dividir, el canal de estímulos, el umbral (*threshold*), el número de estímulos y el tamaño de la ventana de segmentación. Una vez generados, el usuario puede navegar entre los trials resultantes e incluirlos o descartarlos individualmente del conjunto que participará en los análisis posteriores.

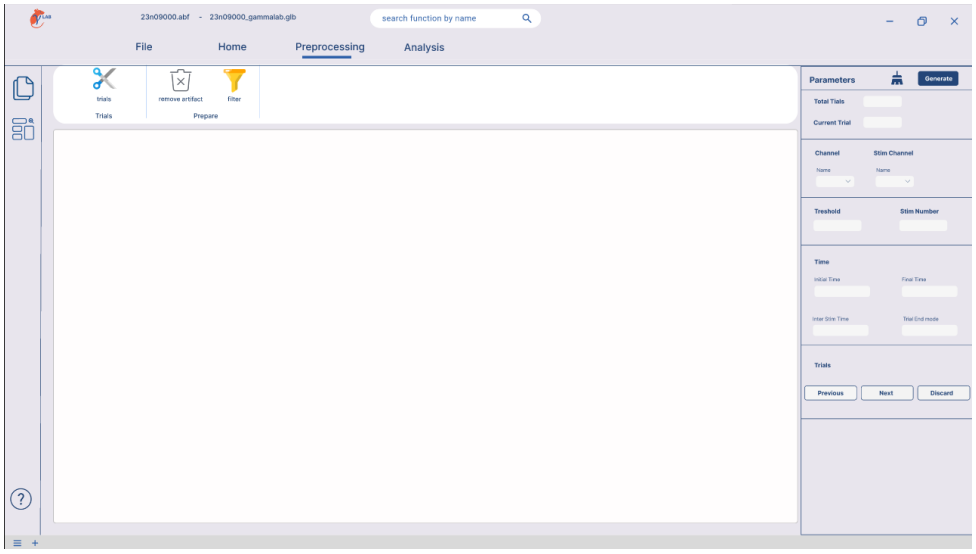


Figura 23: opciones de trials

Categoria *prepare*

- **remove artifact:** elimina artefactos presentes en los trials mediante la selección de un rango de tiempo a corregir, ofreciendo dos modos de corrección: interpolación del segmento seleccionado o eliminación de la señal desde el inicio del trial hasta el punto seleccionado.

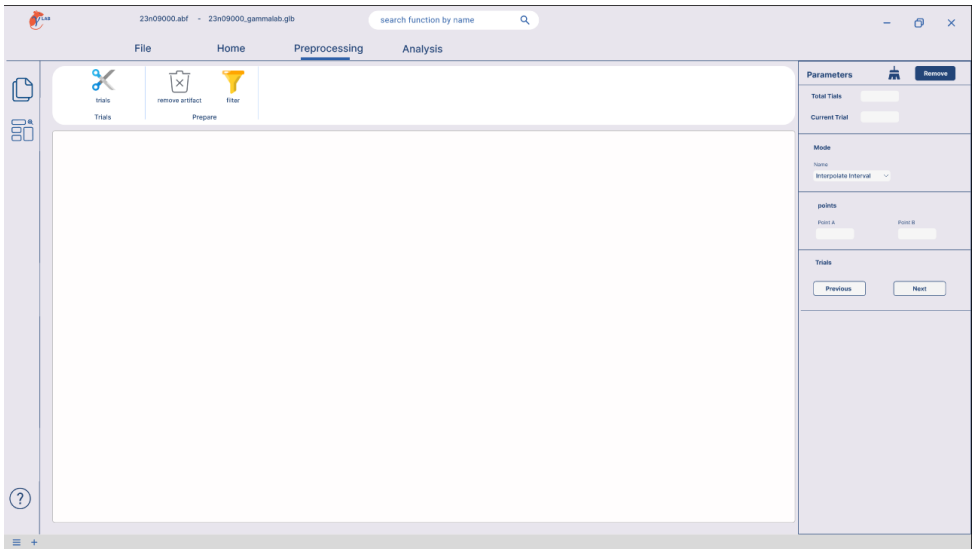


Figura 24: opciones de remove artifact

- **filter:** aplica un filtro digital Butterworth a la señal, mediante la selección del rango de frecuencias de corte y el orden del filtro, mostrando el resultado tanto sobre la señal completa como sobre el primer trial generado.

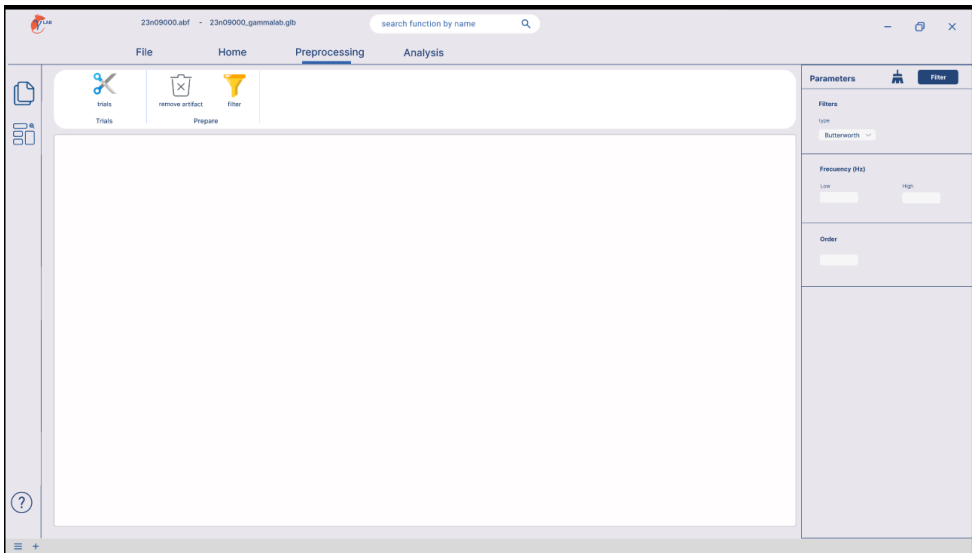


Figura 25: opciones de filter

3.6.5. Analysis

En esta sección se pueden realizar diferentes análisis de la señal, a continuación seran listados los análisis disponibles y sus opciones:

- **FFT:** calcula la transformada rápida de Fourier de la señal, mediante la configuración de la frecuencia de muestreo y el rango de frecuencias de interés, y muestra el espectro de potencia resultante de forma individual para cada ensayo.

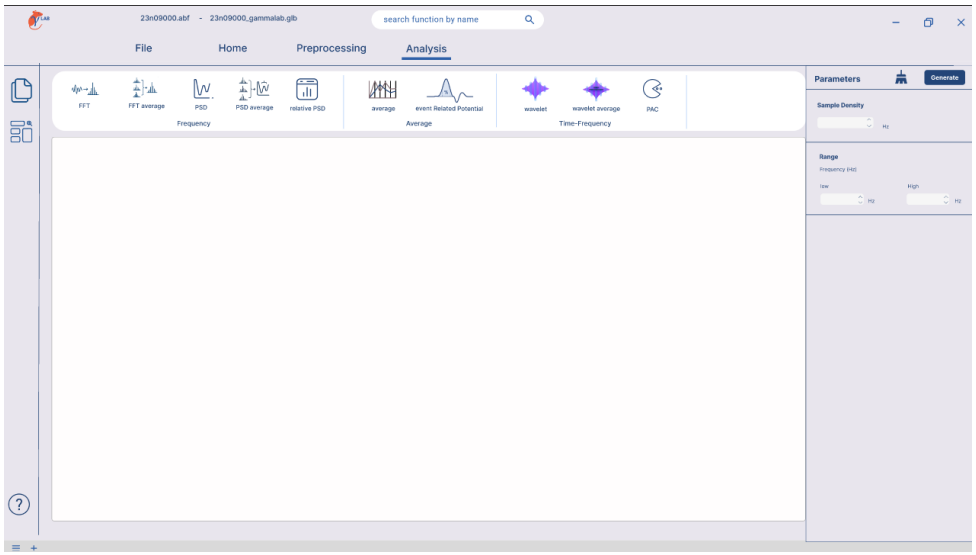


Figura 26: opciones de FFT

- **FFT Average:** calcula la FFT de la señal con la misma configuración de frecuencia de muestreo y rango de frecuencias que la opción anterior, pero adicionalmente promedia los espectros obtenidos de todos los ensayos para reducir el ruido y resaltar los componentes de frecuencia comunes entre ellos.

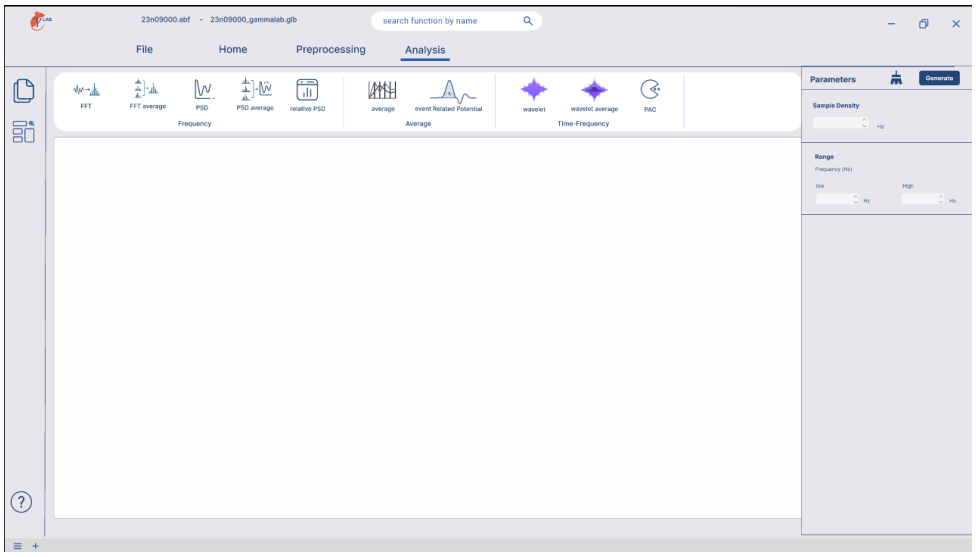


Figura 27: opciones de FFT Average

- **PSD:** estima la densidad espectral de potencia de la señal mediante el método de Welch, permitiendo configurar el modo de cálculo, el tipo de ventana, el rango de frecuencias y el modo de *detrend*, y muestra el resultado de forma individual para cada ensayo.

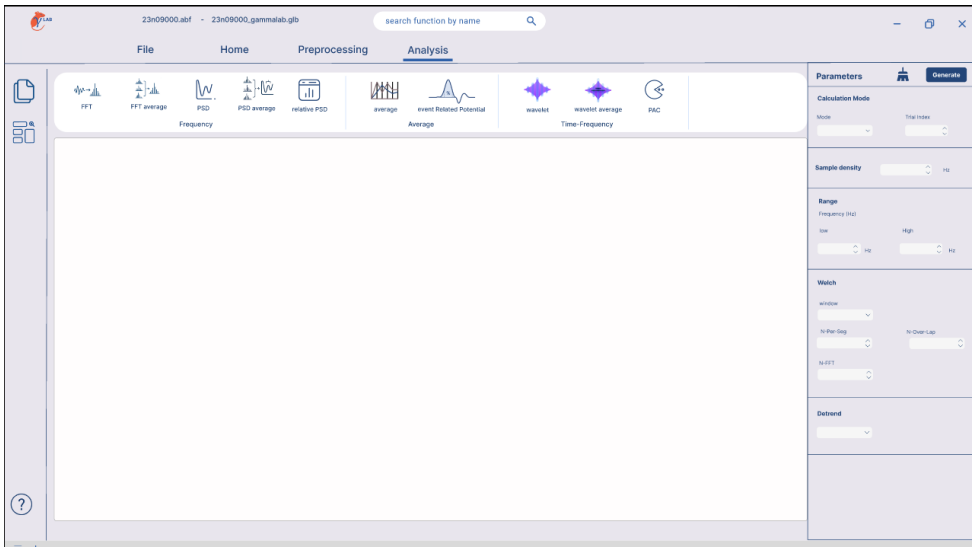


Figura 28: opciones de PSD

- **PSD Average:** estima la PSD de la señal mediante el método de Welch, con la misma configuración de frecuencia de muestreo, ventana y rango de frecuencias que la opción anterior, pero promediando de forma agregada los resultados de todos los ensayos.

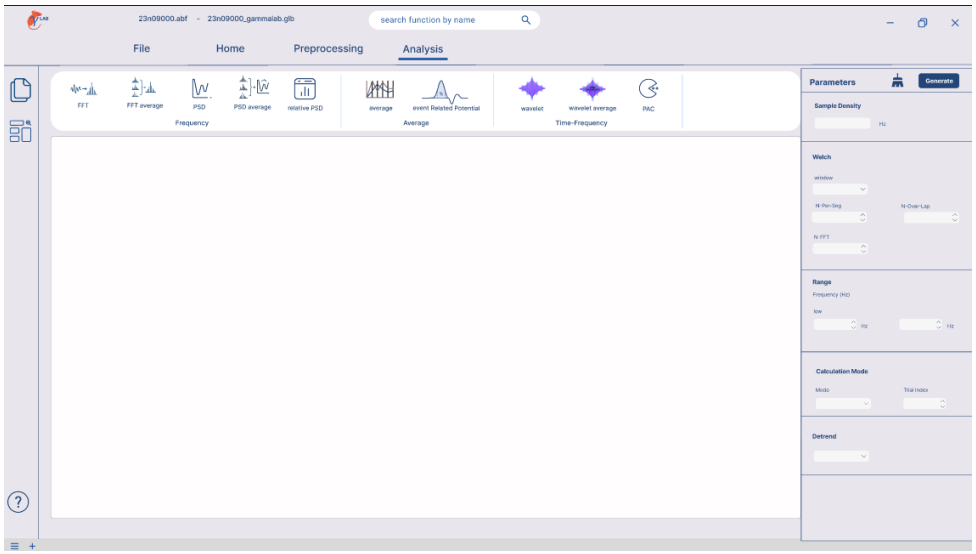


Figura 29: opciones de PSD Average

- **PSD relative:** calcula la PSD relativa de la señal, entregando tanto la potencia absoluta como la potencia relativa aportada por cada banda de frecuencia del rango seleccionado, mediante la configuración del tipo de ventana, el modo de *detrend* y el modo de cálculo.
- **Average:** calcula y visualiza el promedio temporal de los trials generados, útil para observar patrones comunes y atenuar variaciones aleatorias entre ensayos.

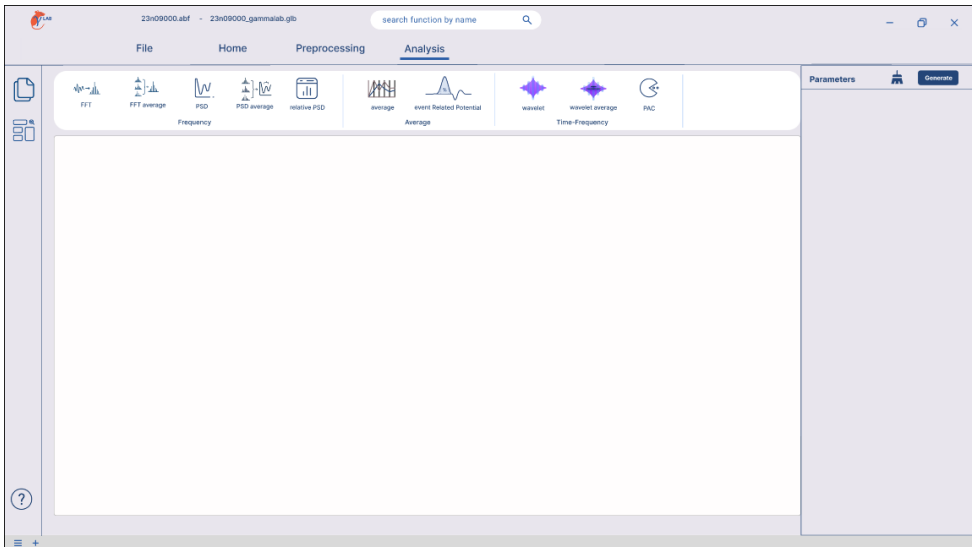


Figura 30: opciones de Average

- **Event related potential:** calcula el potencial relacionado con eventos (ERP), generando un *Butterfly Plot* con todos los trials alineados y un espectrograma de la dinámica tiempo-frecuencia asociada al evento.

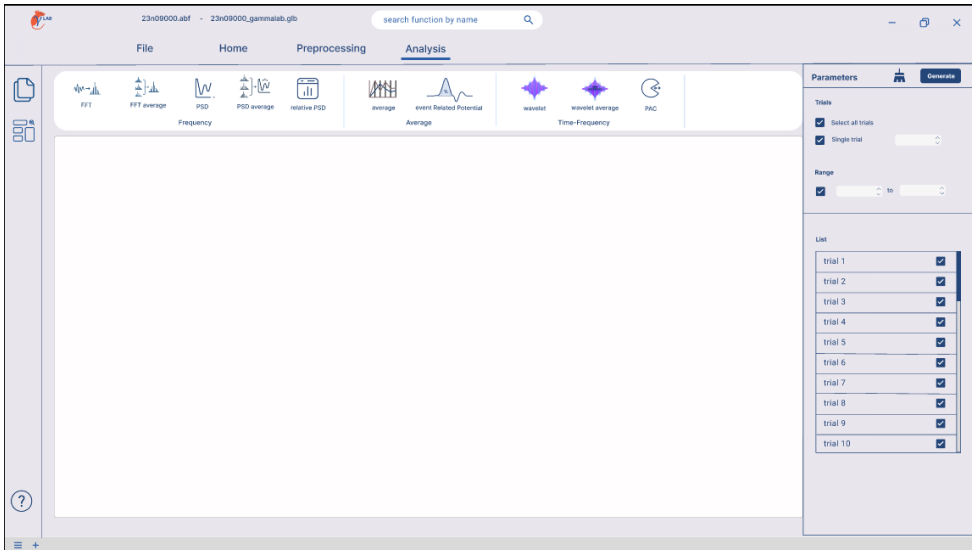


Figura 31: opciones de Event related potential

- **Wavelet:** calcula la transformada wavelet continua de la señal, mediante la configuración de la frecuencia de muestreo, el rango de frecuencias, el número de ciclos, la normalización y la escala de visualización, permitiendo observar cómo varía el contenido espectral de la señal a lo largo del tiempo para cada ensayo.

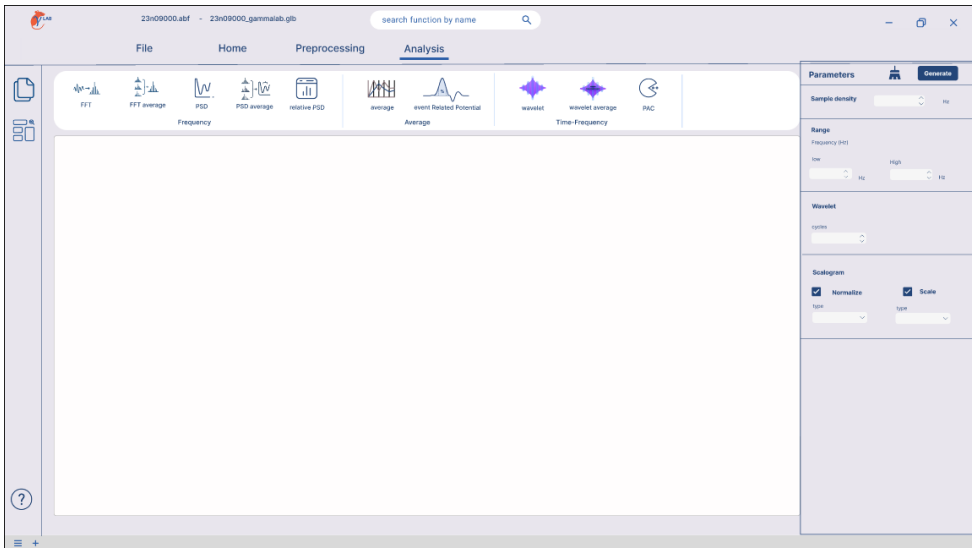


Figura 32: opciones de Wavelet

- **Wavelet Average:** calcula la transformada wavelet con la misma configuración que la opción anterior, y promedia los resultados sobre múltiples trials, generando un mapa tiempo-frecuencia agrupado.

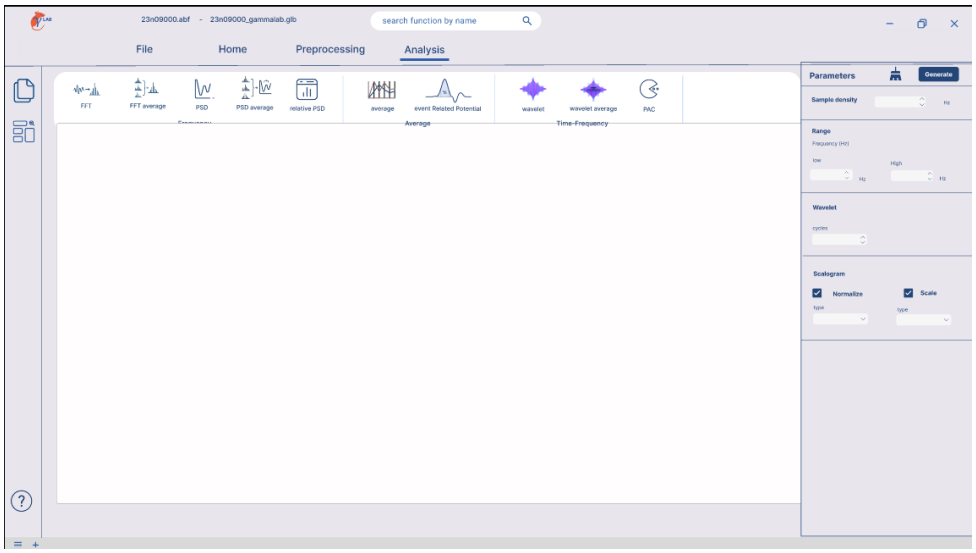


Figura 33: opciones de Wavelet Average

- **PAC:** calcula el acoplamiento de fase-amplitud (*Phase-Amplitude Coupling*) de un canal, evaluando la relación entre la fase de una banda de baja frecuencia y la amplitud de un rango de frecuencias seleccionado, ya sea sobre un trial individual o sobre el promedio de varios trials. El resultado incluye el mapa de acoplamiento fase-frecuencia, el histograma de distribución de amplitud en función de la fase, y el vector de acoplamiento medio con su magnitud y ángulo.

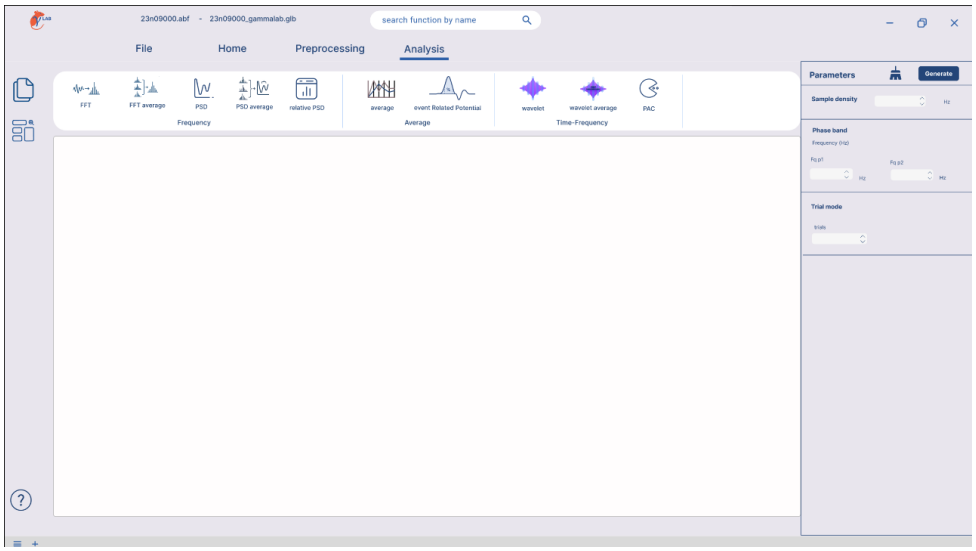


Figura 34: opciones de PAC

4. Comportamiento del sistema

4.1. Diagrama guardar proyecto

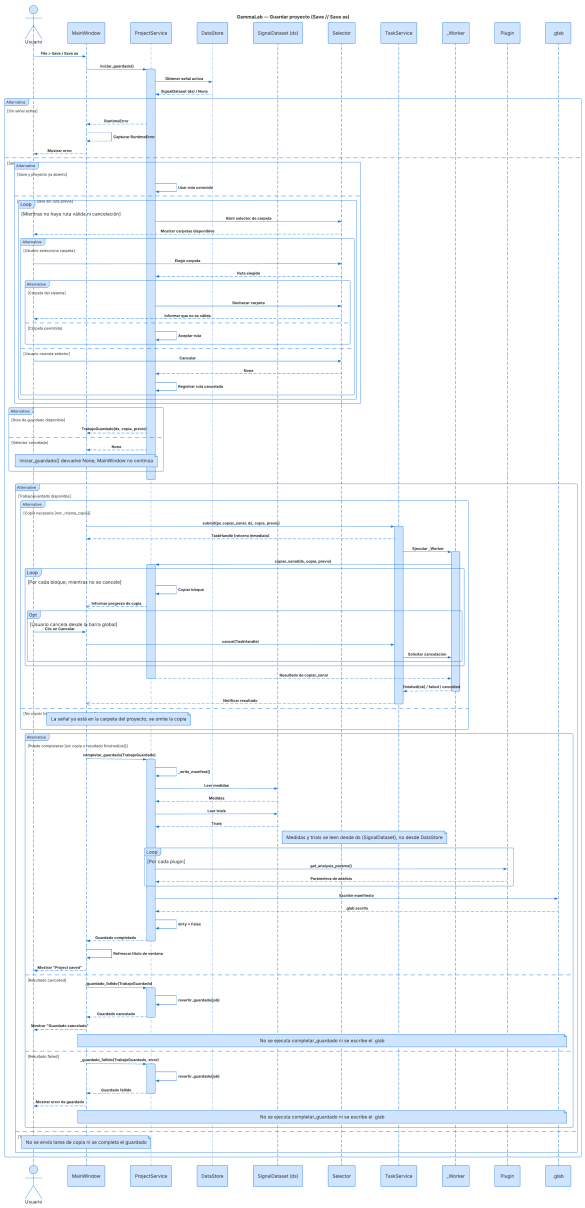


Figura 35: Diagrama de secuencia para guardar proyecto

Este diagrama representa el flujo que sigue Gamma Lab v.20 al momento de usar `save` y `save as` en un proyecto. El proceso comienza cuando el usuario solicita guardar el poryecto desde la interfaz pirncipal `MainWindow`, que llama al método `inicar_guardado` de `ProjectService`.Este servicio verifica que exista una señal activa mediante `DataStore`. Si no hay una señal disponible, se genera un `RuntimeError`, que es capturado por `MainWindow` para informar al usuario y detener el proceso.

Cuando existe una señal cargada, `ProjectService` determina la ruta en la cual se guardará el proyecto. Si la ruta es válida, se utiliza directamente; de lo contrario, se abre el selector de carpetas para que el usuario elija una ubicación. El sistema valida la carpeta seleccionada y, si no

está permitida, informa al usuario y permite realizar una nueva selección. Este ciclo se mantiene hasta que la carpeta seleccionada sea válida o el usuario cancele la selección. En este último caso, `iniciar_guardado` devuelve `None` y `MainWindow` cancela la acción.

Una vez definida la ruta, `ProjectService` devuelve la información necesaria para realizar el guardado. Si la señal requiere ser copiada a la carpeta del proyecto, `MainWindow` envía la tarea a `TaskService`, que ejecuta el trabajo de forma asíncrona mediante `_Worker`. La copia se realiza por bloques y se informa el progreso al usuario. Durante esta etapa, el usuario puede cancelar le proceso desde la barra de progreso global; la solicitud se transmite a `TaskService` y al trabajador encargado de la operación. En caso que la señal ya se encuentra en la carpeta del proyecto, se omite esta tarea de copia.

Cuando la copia finaliza correctamente, o cuando no era necesaria, `MainWindow` llama al método `completar_guardado()` dentro de `ProjectService`. Allí se genera el manifiesto del proyecto, se consultan las medidas y los trials directamente desde `SignalDataset`, y se recorren los plugins para obtener sus parámetros de análisis mediante `get_analysis_params()`. Finalmente, se escribe el archivo `.glab`, se establece `dirty = False` y se notifica a `MainWindow` que el guardado se completó. La interfaz actualiza el título de la ventana y muestra el mensaje de confirmación **"Project saved"**.

En caso contrario, si la copia es cancelada o falla, se ejecuta `guardado_fallido()`, que revierte el estado temporal del guardado y comunica el resultado correspondiente al usuario. En estos casos no se ejecuta `completar_guardado()` ni se escribe el archivo `.glab`. De esta manera, el diagrama muestra que la generación del archivo del proyecto depende de que todas las operaciones previas finalizen correctamente.

4.2. Diagrama cerrar proyecto

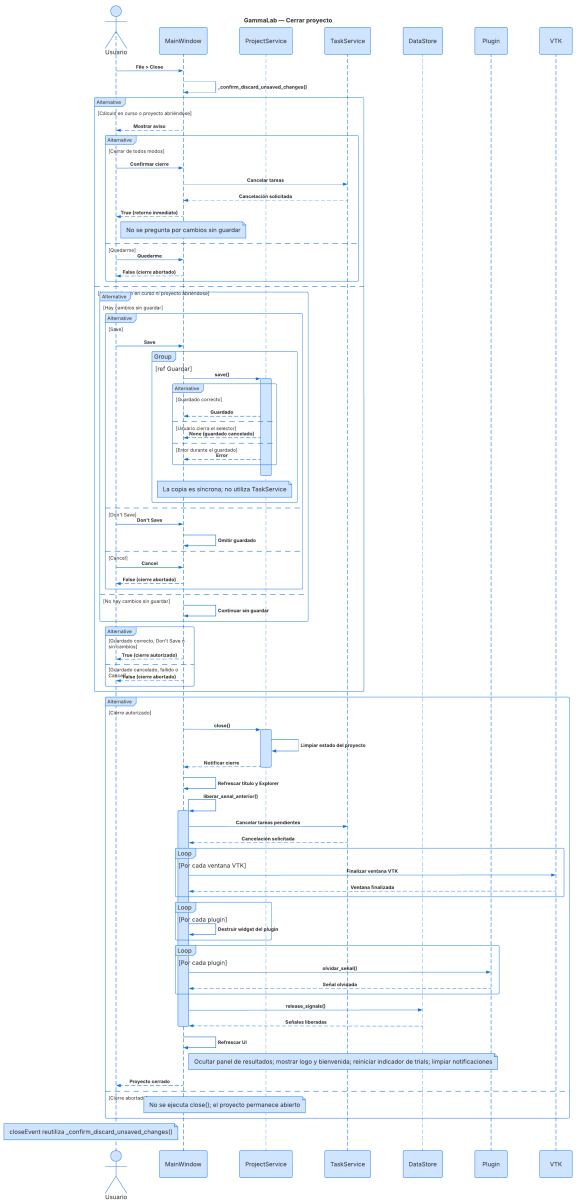


Figura 36: Diagrama de secuencia para cerrar proyecto

Este diagrama representa el flujo que sigue Gamma Lab v.20 al momento que un usuario solicita cerrar un proyecto desde la opción **File >Close**. El proceso comienza en `MainWindow`, que invoca `_confirm_discard_unsaved_changes()` para determinar si el cierre puede realizarse de forma segura y si es necesario solicitar una confirmación al usuario.

En primer lugar, se verifica si hay un cálculo en ejecución o si el proyecto se encuentra en proceso de apertura. Cuando se presenta alguna de estas condiciones, la interfaz muestra un aviso que permite al usuario decidir entre cerrar de todos modos o permanecer en el proyecto. Si el usuario confirma el cierre, `MainWindow` solicita a `TaskService` la cancelación de las tareas y autoriza inmediatamente el cierre. En esta situación no se realiza una consulta adicional sobre los cambios sin guardar. Si el usuario decide quedarse, el método devuelve `False` y el cierre se interrumpe.

Cuando no hay cálculos en ejecución ni procesos de apertura, el sistema verifica si existen cambios sin guardar. Si los hay, presenta las opciones **Save**, **Don't Save** y **Cancel**. Al seleccionar **Save**, `MainWindow` solicita a `ProjectService` ejecutar `save()`. Este guardado se realiza de manera síncrona y no utiliza `TaskService`. Si el guardado termina correctamente, el flujo puede continuar; si el usuario cancela el selector de guardado o se produce un error, el cierre se aborta. Al seleccionar **Don't Save**, se omite el guardado y se permite continuar. En cambio, **Cancel** interrumpe el procedimiento y mantiene el proyecto abierto. Si no existen cambios pendientes, el sistema también puede continuar sin guardar.

Una vez autorizada la operación, `MainWindow` llama a `close()` en `ProjectService`, que limpia el estado del proyecto y notifica el cierre a la interfaz. Posteriormente, se actualizan el título de la ventana y el Explorer. A continuación, `MainWindow` ejecuta `liberar_senal_anterior()` para liberar los recursos asociados a la señal y al proyecto. Este procedimiento comienza solicitando la cancelación de las tareas pendientes mediante `TaskService`. Después, se recorren las ventanas de `VTK` para finalizarlas, se destruyen los widgets de los plugins y se llama a `olvidar_senal()` en cada plugin para desvincularlo de la señal anterior. Finalmente, `DataStore` ejecuta `release_signals()` para liberar las señales almacenadas.

Como última etapa, `MainWindow` actualiza la interfaz: oculta el panel de resultados, muestra el logo y la pantalla de bienvenida, reinicia el indicador de trials y limpia las notificaciones. Una vez terminadas estas operaciones, informa al usuario que el proyecto se ha cerrado.

Si en cualquiera de las validaciones iniciales el cierre es cancelado o no puede completarse, no se ejecuta `ProjectService.close()` y el proyecto permanece abierto. El diagrama también muestra que el evento `closeEvent` reutiliza `_confirm_discard_unsaved_changes()`, manteniendo el mismo mecanismo de confirmación cuando el usuario intenta cerrar la ventana principal.

4.3. Diagrama wavelet

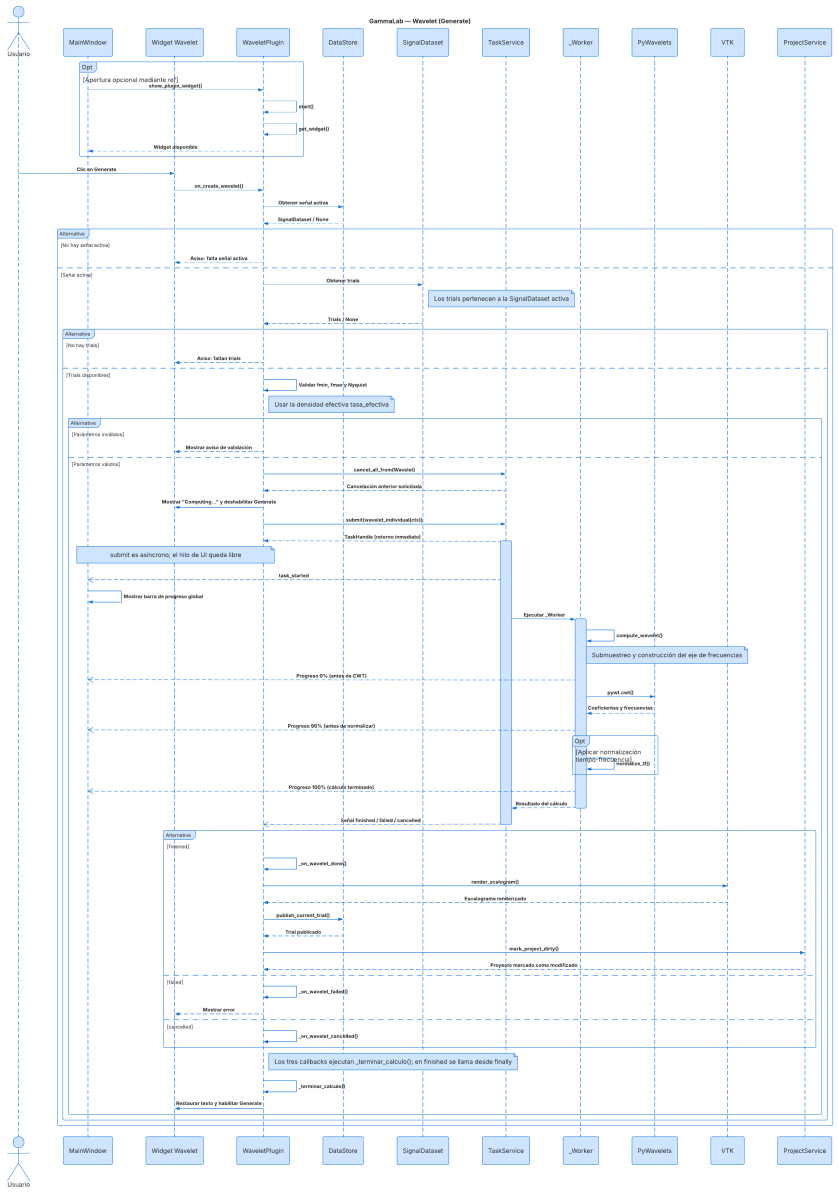


Figura 37: Diagrama de secuencia para wavelet

Este diagrama representa el flujo que sigue Gamma Lab v.20 al momentode generar un análisis con Wavelet a partir de los trial de una señal activa. Este flujo comienza con la apertura del widget de wavelet desde `MainWindow`. Para ello, se solicita al `WaveletPlugin` que inicialice el plugin y proporcione su interfaz mediante `get_widget()`. Una vez disponible el widget, el usuario puede seleccionar los parámetros del análisis y ejecutar la opción **Generate**.

Al iniciar la generación, el widget invoca `on_create_wavelet()` en `WaveletPlugin`. El plugin consulta la señal activa y verifica que exista un objeto `SignalDataset`. Si no hay una señal activa, se muestra una advertencia y el proceso se detiene. Si existe, se obtienen sus trials directamente desde `SignalDataset`, ya que estos pertenecen a la señal activa. Cuando no hay trials disponibles, también se informa al usuario y no se continúa con el cálculo.

Si existen trials, el plugin valida los parámetros de entrada, incluyendo los límites de frecuencia (`fmin` y `fmax`) y la frecuencia de Nyquist. Para esta validación se utiliza la densidad de muestreo

efectiva (`tasa_efectiva`). Si los parámetros no son válidos, se muestra el aviso correspondiente. En caso contrario, el plugin solicita la cancelación de cálculos anteriores relacionados con Wavelet mediante `TaskService`, cambia el estado del widget a **"Computing..."** y deshabilita el botón **Generate** para evitar que se inicie otra ejecución mientras el cálculo está en curso.

A continuación, `WaveletPlugin` envía el cálculo a `TaskService` mediante `submit (wavelet_individual (ctx))`. Este servicio devuelve inmediatamente un `TaskHandle` y ejecuta el trabajo de forma asíncrona a través de `_Worker`, permitiendo que el hilo de la interfaz permanezca libre. El trabajador realiza el cálculo Wavelet mediante `compute_wavelet ()`, que utiliza PyWavelets (`pywt.cwt`) para obtener los coeficientes y las frecuencias correspondientes. De acuerdo con la configuración del análisis, también puede aplicarse una normalización en el dominio tiempo-frecuencia mediante `normalize_tf ()`.

Durante la ejecución se actualiza el progreso global: se informa un avance inicial antes de ejecutar la transformada, un avance aproximado del 90 % antes de la normalización y el 100 % cuando el cálculo termina. Al finalizar, `TaskService` comunica el resultado al plugin mediante una de las señales disponibles: `finished`, `failed` o `cancelled`.

Cuando el cálculo finaliza correctamente, se ejecuta `on_wavelet_done()`. El plugin solicita a `VTK` la representación del escalograma mediante `render_scalogram()`, publica el trial generado a través de `publish_current_trial()` y notifica a `ProjectService` que el proyecto ha sido modificado mediante `mark_project_dirty()`. Si ocurre un error, `on_wavelet_failed()` se encarga de mostrar el mensaje correspondiente; si el usuario cancela la operación, `on_wavelet_cancelled()` gestiona ese resultado.

Finalmente, los tres callbacks ejecutan `_terminar_calculo()` para restaurar el estado del widget, recuperar el texto original del botón y habilitar nuevamente la opción **Generate**. Así, el diagrama representa tanto el procesamiento exitoso como las rutas de validación, error y cancelación, mostrando la interacción entre la interfaz, el plugin, el sistema de tareas, las herramientas de procesamiento y la visualización.